MedPharm ERP

DEVELOPER MANUAL

The source-of-truth reference for the engineers who build, extend, and maintain MedPharm ERP.

VOLUME III

Revision 1.7.6-D // Issued May 2026

Enlightec Ltd. · Medical & Pharmaceutical Enterprise Resource Planning

Document Control

Field	Value
Title	MedPharm ERP — Developer Manual
Volume / Edition	Volume III — Engineer Edition
Revision	1.7.6-D
Issue Date	27 May 2026
Author	Robert Andrew Stillwell
Publisher	Enlightec Ltd., www.enlightec.com
Applies To	MedPharm ERP versions 1.7.x (latest 1.7.6)
Classification	CONFIDENTIAL — Engineering Internal
Licence	GNU General Public License v3.0
Companion Volumes	Volume I — Technical Reference (MedPharm_ERP_Documentation.pdf) Volume II — Servi
Source Repository	https://github.com/stillwell/MedPharm
Container Registry	https://hub.docker.com/u/enlightec
Support Contact	Andrew.Stillwell@enlightec.com

Document Purpose

This manual is the authoritative description of MedPharm ERP's software construction. It answers the questions an engineer has when the code is unfamiliar: *why does it look like this, where do I put my change, how do I test it, and what will I break?* Wherever possible, answers are grounded in code samples pulled from the current tree; wherever the code cannot answer on its own, the manual provides the surrounding rationale that would otherwise live in a senior engineer's head.

Three readers are kept in mind throughout. The **new hire** reads cover-to-cover over the first week and returns to specific chapters thereafter. The **returning maintainer** skims the table of contents for the module they need to change and consults only the relevant chapter. The **external auditor** reads Parts II through V to evaluate whether the codebase implements the clinical, financial, and compliance claims made in the product marketing material. This document should serve all three

without qualification.

Contacting the Authors

For correction, clarification, or extension requests, write to Andrew.Stillwell@enlightec.com. Responses arrive within two business days in all but exceptional weeks; if you have not heard back within four business days, please resend with the subject line prefixed with `[devmanual-followup]`.

Bug reports that are not confidential are welcome as GitHub issues at https://github.com/stillwell/MedPharm/issues. Confidential reports — including suspected security vulnerabilities — must go by email only; see Chapter 30 for the responsible disclosure process in full.

CHAPTER

Foreword

A piece of medical software is never finished. It accumulates improvements, absorbs regulatory changes, and outlasts the people who first shaped it. The MedPharm ERP codebase has already lived through more hands than one engineer could keep in memory; this manual is the artefact that allows the next hand to pick up where the previous one set the tools down.

Two manuals already attend to adjacent needs. Volume I (Technical Reference) describes the system as a finished artefact: its models, its routes, its contracts. Volume II (Service Manual) describes the system under an operator's care: its commissioning, its daily routines, its failure modes. Volume III is the inside-out companion to those two. It describes the system as a thing that was *made* and can be *made further*. Where the two earlier volumes assume the code is settled, Volume III assumes — correctly — that it is not.

The manual is written in the second person. When it says "you", it is addressing the engineer at the keyboard; when it says "we", it is describing a decision the project has made, not a royal plural. The tone is plain, the sentences are intentionally unadorned, and the code samples are real. Where a diagram helps, a diagram is drawn. Where a line of SQL is clearer than three paragraphs of prose, the SQL appears without apology.

Nothing in the pages that follow is closed to revision. If you find an inaccuracy, a gap, or an outright lie that the code has since corrected, please write in and the manual will be updated. A revision letter (1.7.6-A, 1.7.6-B, …) is appended for each re-issue, and a visible record appears in Appendix F.

Robert Andrew Stillwell
Enlightec Ltd.

CHAPTER

How to Use This Manual

The manual is written to support three reading strategies. A reader who adopts any one of them should find the pages cooperative.

Cover-to-cover as onboarding

A newly hired engineer should expect to spend three to five working days reading the manual end to end, pausing at each chapter to open the real files on disk and trace the code against the narrative. The parts are ordered with this path in mind — foundations first, then data, then backend, then clients, then cross-cutting concerns, then workflow. By the end of Part IV the reader has a working mental map of every user-facing surface; Parts V through VII are the operational cement that holds the surfaces to the backend.

As a reference

Once a reader has read the manual once, the table of contents becomes the primary index. Chapters are self-contained where possible; where a chapter depends on earlier material, the dependency is stated in the opening paragraph. The appendices — in particular the API endpoint catalogue and the file-by-file code tour — are designed to answer the "where does the code live for X?" question in under a minute.

Under change pressure

If you are about to land a substantial change — a new model, a new endpoint, a new client platform — consult Part VII (Extending MedPharm) before you start. The recipes there are written to prevent the specific mistakes that the project has already been burned by, and following them will save you from rediscovering those mistakes at code review.

A note on completeness

This document does not claim that every line of code is explained. It does claim that every *non-obvious decision* is explained, and that a reader who has absorbed the manual can learn the rest by reading the code. Where you find yourself unable to bridge the gap between what the manual says and what the code does, treat that as a bug report and file it.

CHAPTER

Conventions and Signal Words

Consistent typographic and editorial conventions are used throughout. Scanning a page should give a reliable first impression of its density and seriousness before a full read.

Typography

Identifiers, filenames, environment variables, and HTTP routes appear in a monospaced navy font — for example `DatabaseManager`, `api/routes.py`, `MEDPHARM_JWT_SECRET`, or `POST /api/v1/patient/messages`. Longer listings are set in a dark-backgrounded code block with line numbers, and source language is indicated in the accompanying caption where it is not obvious from context.

Cross-references to other volumes appear as **Vol. I § 4.3** or **Vol. II Ch. 19**. Internal cross-references use the same shorthand without the volume prefix — **Ch. 14** means Chapter 14 of this manual.

Hyperlinks are indigo and underlined — for example https://github.com/stillwell/MedPharm or Andrew.Stillwell@enlightec.com. Hyperlinks are clickable in PDF viewers that support them (Adobe Acrobat Reader, Apple Preview, and most modern browsers do).

Signal Words

Four signal-word boxes appear throughout. Their meanings are precise; they are never substituted for emphasis.

Note. A Note conveys useful but non-critical context. Skipping one will not cause harm; reading one often saves time.

Tip. A Tip offers a non-obvious technique or shortcut that has been useful to previous engineers. Tips are optional but usually worth the thirty seconds they save.

Caution. A Caution introduces information that, if ignored, will cause degraded behaviour or avoidable rework. No permanent damage is expected, but the affected engineer will wish they had read more carefully.

Danger. A Danger introduces information that, if ignored, will cause data loss, security compromise, or other outcomes from which recovery is costly. Do not proceed past a Danger without understanding why it applies.

Verbs of Requirement

The manual follows the common engineering convention under which **must** indicates a firm requirement, **should** indicates a strong recommendation whose violation must be justified in writing, and **may** indicates a permissive option. **Must not** and **should not** mirror their positive forms.

Time and Date Notation

All timestamps in log excerpts and examples use ISO 8601 with a timezone offset; when no offset is shown, UTC is assumed. Developers should configure their local machines to log in UTC — it simplifies correlation across the team and matches the convention used in production.

Table of Contents

PART

I

Foundations

A new engineer's orientation — what MedPharm is, why it was built the way it is, and how its pieces fit together.

CHAPTER 1

Project Vision and Problem Space

MedPharm ERP exists to fold the operational workload of a small-to-medium medical practice or retail pharmacy into a single coherent system. The alternative — which most such organisations live with — is a constellation of three to five uncoordinated applications: a patient record system that cannot speak to the billing system, a billing system whose insurance claims are exported by CSV to a third party, a pharmacy formulary that lives in a binder, and a scheduling tool whose access permissions are a spreadsheet in someone's inbox. Each of these gaps is a place where patient care or revenue quietly leaks away.

The product's ambition is therefore not cleverness but coherence. It does not attempt to out-feature the market leaders; instead it attempts to do a modest feature set well, on modest hardware, in a single SQLite database that an operator can back up with `cp`. Every design decision in this manual should be read against that backdrop. Where a feature could be implemented in two ways and one of them requires a second daemon, the project chooses the one that does not.

Who uses MedPharm

Three classes of user interact with the running system. Clinical staff — doctors, psychiatrists, nurses, pharmacists, office managers, and administrators — use the PyQt6 desktop application as their primary workstation. Patients use the Flask web portal on a browser, or one of the native mobile clients (Android, iOS) or desktop clients (macOS, Windows) that connect to the cloud REST API. Operators and system administrators interact with the system through log files, the `./install.sh` wrapper, Docker compose, and occasional direct SQL.

The four user-facing surfaces on the patient side share one backend. The clinical desktop application is unique in that it may bypass the HTTP API and speak directly to the database through the in-process DatabaseManager — a deliberate choice to keep clinical latency low when the backend runs on the same machine. In the zero-configuration default that database is a local SQLite file; where `MEDPHARM_DATABASE_URL` points every surface at a shared PostgreSQL server, the desktop speaks to that same database directly instead. Chapter 3 expands on this topology, and Chapter 17 discusses its implications for feature development.

What MedPharm is not

It is useful to state the project's non-goals explicitly so that an engineer does not waste effort implementing the wrong thing in the right way. MedPharm is *not* a hospital information system. It does not drive bedside monitors, does not negotiate with radiology PACS, and does not implement HL7 v2 messaging. It is *not* a multi-tenant SaaS platform — a single database serves a single practice, and any attempt to run several databases through one Flask worker is unsupported. It is *not* a research platform — the schema reflects the needs of clinical operation, not statistical research, and bulk

de-identified exports are out of scope.

Where the name comes from

"MedPharm" is the portmanteau of *medical* and *pharmaceutical*. The project was scoped from the outset to serve both clinic-side (patient care, scheduling) and pharmacy-side (dispensing, drug interactions, NDC code management) workflows within one data model. This is why the Medication and Prescription models are first-class citizens rather than appendages — they predate some of the clinical tables by several design iterations.

Regulatory context

MedPharm stores protected health information and is therefore subject to HIPAA's Privacy Rule (45 CFR Part 164, Subpart E) and Security Rule (Subpart C). The engineering implications are far-reaching: role-based access control, audit logging of every PHI touch, encryption at rest for sensitive fields, transmission security over TLS, breach notification procedures, and a Security Officer role. The specific models and code paths that support each of these requirements are catalogued in Chapter 9; the broader compliance posture is the subject of the companion **HIPAA_COMPLIANCE.md** document in `docs/`.

> **Note.** HIPAA is the baseline. Deployments outside the United States may be subject to GDPR (European Union), PIPEDA (Canada), or comparable local regimes. The code is written defensively enough that most of these regimes can be satisfied with configuration rather than code changes, but the work of mapping the controls is the deploying organisation's responsibility.

A brief history

The project began as a PyQt proof-of-concept in 2024, acquired a Flask web portal a month later, grew a Cloud REST API when mobile clients were added, and reached multi-platform parity with the 1.7.x line in early 2026. The HIPAA-compliance features, TLS-by-default, and secure messaging were folded in during the 1.6 → 1.7 transition. The most recent substantial addition is the private provider-notes feature described in Chapter 10.

CHAPTER 2

Technology Stack and Its Rationale

Every technology choice in MedPharm answers to three constraints: it must be operable by a single System Operator, it must store PHI safely, and it must be buildable in a container in under ten minutes. Those three constraints alone eliminate most of the fashionable options of the last decade. What survives is a short list of tools that favour clarity, longevity, and first-party documentation.

The backend

Layer	Tool	Version	Rationale
Language	Python	3.10+	Mature type hints, wide library surface, engineering staff overlap with
Web framework	Flask	3.0+	Minimal; readable routing table; no baked-in assumptions that conflict
ORM	SQLAlchemy	2.0+	Mapped-dataclass style, clear session lifecycle; the same mappings c
Database	SQLite (default) / PostgreSQL	3.40+ / 14+	SQLite is the zero-config default — one file, no daemon, WAL concur
Password hashing	Werkzeug / PBKDF2-SHA256	3.0+	Deterministic, FIPS-track, ships with Flask ecosystem.
Symmetric encryption	cryptography / Fernet	46+	AES-128-CBC + HMAC-SHA256, industry-standard authenticated en
JWT signing	stdlib hmac + hashlib	—	Handwritten to avoid third-party JWT libraries' key confusion vulnerab
WSGI server	gunicorn (gevent worker)	21.2+	Battle-tested, simple configuration, ships in Ubuntu 24.04 repositories
Reverse proxy	Nginx	1.24+	TLS termination, HTTP/2, tight request-size limits, wide operator fami

The clients

Platform	Language	Frameworks	Target
Clinical desktop	Python 3.10+	PyQt6	Any OS with a display server
Web portal (patients)	Python 3.10+	Flask + Jinja2	Modern browsers; Bootstrap 5 UI
Android (patients)	Kotlin 1.9+	MVVM, Retrofit, OkHttp, Coroutines, EncryptedSharedPreferences	API level 26+ (Android 8.0+)
iOS (patients)	Swift 5.9+	SwiftUI, async/await, Keychain Services	iOS 16+
macOS	Swift 5.9+	SwiftUI, NavigationSplitView	macOS 13+
Windows	C# / .NET 8	WPF, CommunityToolkit.Mvvm, DPAPI	Windows 10 1809+

Why not newer, more fashionable choices

The project deliberately declined several popular options. Django was considered in place of Flask but rejected for its size — most of Django is unused here, and its ORM would have duplicated SQLAlchemy rather than cooperated with it. PostgreSQL is now a first-class backend rather than a hypothetical one: setting `MEDPHARM_DATABASE_URL` to a `postgresql+psycopg://…` DSN points every surface at one shared server, which is the configuration to reach for the moment a deployment outgrows a single writer. SQLite nonetheless remains the default precisely because at a small clinic's load — perhaps two writes per second at peak — its single-writer limitation is not the bottleneck, and the operational simplicity of one file an operator can copy is worth a great deal. Redis for session storage was considered and rejected because Flask's default signed-cookie session is sufficient and Redis would be another daemon to monitor.

On the client side, Flutter and React Native were both rejected in favour of native UI toolkits on each platform. The reasoning is not aesthetic — cross-platform toolkits can ship acceptable UI — but operational: a native client in SwiftUI or Jetpack Compose will run correctly on devices that the cross-platform runtime has already abandoned, and medical device fleets are notoriously long-lived. The engineering cost of maintaining four separate codebases is the price the project pays for that longevity.

Operating system targets

The backend is supported on Ubuntu Server 24.04 LTS as the primary target and on Fedora, RHEL, Debian, Arch, and macOS as best-effort secondary targets. The Docker images are built on Ubuntu 24.04 LTS. The install script detects the operating system and adjusts package manager invocations accordingly; see Chapter 26 for details.

CHAPTER 3

Architecture Overview

MedPharm's architecture is a hub-and-spoke with one privileged local spoke. The hub is a Flask REST API (`/api/v1/*`) backed by the SQLAlchemy-managed database — a local SQLite file by default, or a shared PostgreSQL server when `MEDPHARM_DATABASE_URL` is set. Five of the six user-facing surfaces reach the hub over TLS. The sixth — the PyQt desktop — bypasses the HTTP layer entirely and speaks to the same database through the in-process DatabaseManager. This diagram is worth memorising.

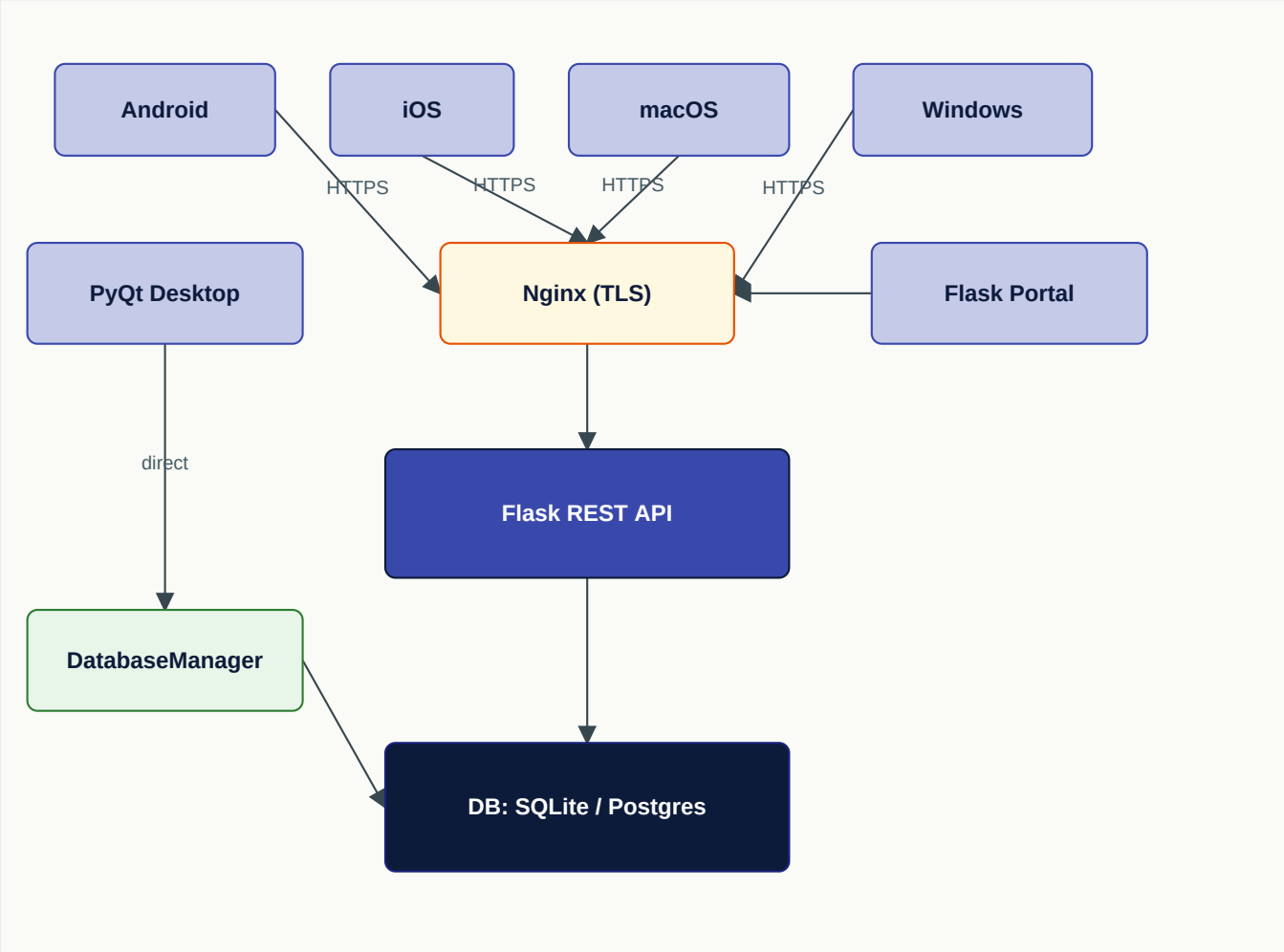

Fig. 3-1. Deployment topology. Solid arrows are synchronous calls; the DatabaseManager is in-process on the clinical desktop.

Layers of the backend

Inside the Flask process, a request travels through four discrete layers. Each layer has a clear responsibility and a single upstream dependency.

- **Blueprint routing** — `api_bp` in `api/routes.py` maps URL patterns to handler functions. This layer is thin by design; it parses request bodies, applies auth decorators, and returns JSON.
- **Authentication** — `@token_required`, `@patient_required`, `@staff_required` decorators in `api/auth.py` verify the bearer token and populate `g.current_user_type`, `g.current_user_id`, and related request-scoped variables.
- **Business logic** — the body of each route function calls methods on `g.db_manager` (attached to Flask's request-scoped `g` object by the application factory).
- **Persistence** — the `DatabaseManager` opens a SQLAlchemy session, performs the reads or writes, and commits. Sessions never outlive the request.

Request lifecycle — a worked example

Consider a patient opening their prescription list in the Android app. The following sequence occurs:

- The Android `PrescriptionsViewModel` calls `repository.getPatientPrescriptions(null)`, which invokes the Retrofit interface method `api.getPrescriptions(null)`.
- OkHttp adds the `Authorization: Bearer …` header via `AuthInterceptor` and performs an HTTPS GET to `/api/v1/patient/prescriptions`.
- Nginx terminates TLS and proxies the request to `gunicorn` on the loopback interface.
- Flask dispatches to `patient_prescriptions()` in `api/routes.py`, whose `@patient_required` decorator verifies the JWT and populates `g.current_patient_id`.
- The route calls `g.db_manager.get_prescriptions_by_patient(pid)`, which opens a session, eagerly loads related `PrescriptionItem` and `Medication` rows, and returns a list of dicts.
- The response is serialised with `flask.jsonify` and returned through the inverse path.

The DatabaseManager as seam

Two components talk to the DatabaseManager: the REST API and the PyQt desktop. Neither component cares about SQL or about how sessions are scoped — they both see an object with a long list of methods like `get_patient_full(patient_id)`, `create_prescription(...)`, and `process_insurance_payment(...)`. This is the single most important architectural seam in the project. Business logic added in the DatabaseManager benefits both surfaces simultaneously; conversely, business logic added only in a route function or only in a Qt widget will skew the two surfaces apart over time. Chapter 11 discusses this at length, and Chapter 31 formalises the pattern as a recipe.

Tip. When in doubt, put the logic in the DatabaseManager. If it turns out that only one surface needed it, the cost is one unused method; if both surfaces needed it, the cost of having put it in only one of them is a subtle divergence that someone has to untangle later.

CHAPTER 4

Repository Layout and File Inventory

A newly checked-out repository has a root directory that looks busier than it is. Most of the files at the top level are launchers, installers, or container metadata; the real code lives in a small number of subdirectories. This chapter provides the map.

Top-level files

File	Purpose
run_qt.py	Clinical desktop application entry point.
run_web.py	Flask web portal entry point.
run_cloud.py	Flask REST API entry point.
install.sh	OS-detection installer. Handles source install, virtualenv, Docker-only, or full-stack Docke
uninstall.sh	Reverses everything install.sh creates. Never touches tracked source files.
start_desktop.sh	Quick-launch wrapper for the Qt app.
start_web.sh	Quick-launch wrapper for the Flask portal.
start_cloud.sh	Quick-launch wrapper for the API server.
start_docker_hub.sh	Interactive Docker Hub launcher — pull, start, stop, choose tag.
generate_docs.sh	Regenerates the Volume I PDF. See also docs/generate_service_manual.py and docs/g
requirements.txt	Qt + web portal dependencies.
requirements-cloud.txt	Cloud API dependencies.
Dockerfile	API-only image (Ubuntu 24.04 base).
docker-compose.yml	Build-from-source Docker Compose file.
docker-compose.hub.yml	Pull-from-Docker-Hub Compose file.
LICENSE	GNU General Public License v3.0.

Top-level directories

Directory	Contents
database/	models.py (all ORM classes and enums), db_manager.py (the DatabaseManager), seed_data.py, seed_e
api/	app.py (application factory), auth.py (JWT + decorators), routes.py (all /api/v1/* endpoints), fhir.py (FHIR F
qt_app/	main_window.py, styles.py, dialogs/login_dialog.py, and widgets/ containing one widget per navigation de
web/	app.py, routes.py, static/ (CSS, JS, vendored assets), templates/ (Jinja2 HTML).
security/	encryption.py (Fernet), audit.py, csrf.py, lockout.py, sessions.py, passwords.py, totp.py, phi.py, emergenc
android/	Gradle project. Kotlin sources under app/src/main/java/com/enlightec/medpharm/.
ios/, macos/	Xcode projects. Swift sources under MedPharm/MedPharm/{Models,Services,Views}/.
windows/	.NET 8 WPF project. Sources under MedPharm/.
server/	Full-stack Docker package — Dockerfile, supervisord.conf, nginx/ configs, entrypoint.sh.
k8s/	Kustomize manifests with cloud-specific overlays (gcp, aws, generic). medpharm-k8s.sh wrapper.
docs/	Three PDF generators (generate_pdf.py, generate_service_manual.py, generate_developer_manual.py) a
.github/workflows/	docker-publish.yml — builds and pushes container images on tagged releases.

Reading order for a new engineer

If you are opening the repository for the first time, the following order will map the territory fastest. Each file sets up the next.

- `database/models.py` — all of the data the system stores.
- `database/db_manager.py` (skim) — methods grouped by domain; no need to read every CRUD, just note the grouping.
- `api/app.py` — the application factory. Shows how security middleware is wired and where `g.db_manager` comes from.
- `api/auth.py` — the JWT implementation and the three access decorators.
- `api/routes.py` — skim; note the blueprint structure and that most routes are one or two lines of logic over a DatabaseManager call.
- `qt_app/main_window.py` — the window's navigation skeleton. Each widget is independently readable.
- `web/routes.py` — the patient portal. Flask-session based rather than JWT.
- `install.sh` — the operator's front door. Reading this file clarifies how the pieces fit together.

Tip. Every one of those files has been kept readable at human scale — the largest is under a thousand lines. If a file starts creeping past 1,500 lines during your work, that is a signal to break it up, not a reason to keep adding.

PART

II

The Data Model

Every table the system stores, why it looks the way it does, and how the models relate to each other in code and in the runtime graph.

CHAPTER 5

Database Design Philosophy

Before any individual model is examined, it is worth articulating the design axioms that produced them. An engineer who understands why the schema is shaped the way it is will make fewer mistakes when extending it.

Axiom 1 — One source of truth

Every fact about a patient, prescription, or invoice lives in exactly one column on exactly one table. When a fact must be derived (age from date of birth, balance due from total minus payments), it is computed at read time or exposed as a Python property on the model. Denormalised copies exist only for legitimate performance reasons and are marked as such in code comments.

Axiom 2 — Soft delete over hard delete

Clinical records must not vanish. Patient rows are deactivated rather than deleted (`is_active=False`); prescriptions are cancelled rather than removed; invoices are voided. Hard deletes are reserved for two cases: transient cache-like rows (failed login attempts, expired TOTP challenges), and rows that never contained PHI (session tokens in the Flask session store).

Axiom 3 — Enums for finite state

Any column whose values are constrained to a small set is modelled as a Python `enum.Enum` subclass and stored via SQLAlchemy's `Enum` type. Eighteen such enums appear in the schema. Using native strings in new code is an anti-pattern that has been systematically removed; do not reintroduce it.

```
class PrescriptionStatus(enum.Enum):
 PENDING = "pending"
 ACTIVE = "active"
 FILLED = "filled"
 EXPIRED = "expired"
 CANCELLED = "cancelled"


class Prescription(Base):
 __tablename__ = "prescriptions"
 id = Column(Integer, primary_key=True, autoincrement=True)
 status = Column(Enum(PrescriptionStatus),
 default=PrescriptionStatus.PENDING,
 nullable=False)
 # … other fields …
```

Listing 5-1. Finite-state columns use Python enums, never raw strings.

Axiom 4 — Relationships are explicit

Every `ForeignKey` is accompanied by an explicit `relationship()` on both sides. Lazy loading is the default; eager loading is opt-in per query via `joinedload()` or `selectinload()`. The reason for this explicitness is defensive: N+1 query problems are easy to introduce and hard to spot in clinical software, so the project prefers predictable lazy-by-default with visible eager decisions to the inverse.

Axiom 5 — PHI is named, tracked, and encrypted when it cannot be keyed

Columns that contain protected health information are catalogued in `security/phi.py`. Where a PHI column must be queryable (patient name, date of birth), it is stored as plaintext and protected by access control and audit logging. Where a PHI column is free-form and unlikely to be queried (provider-note bodies, secure message contents), it is stored as Fernet ciphertext. Chapter 12 gives the cryptographic details.

Axiom 6 — Timestamps in UTC

All timestamp columns default to `datetime.utcnow()`. Timezone conversion is the client's concern. This axiom was violated exactly once in the project's history, discovered during an incident, and the resulting cleanup took long enough that it will not be violated again.

Axiom 7 — Migrations are schema evolutions, never data pivots

SQLAlchemy's `Base.metadata.create_all()` is used for schema creation. Migrations of existing deployments are currently performed manually — SQLite's limitations on `ALTER TABLE` make automated migration fragile. If you are adding a column to an existing table, produce both a model-level change and a one-off migration SQL file in `docs/migrations/` with the convention `YYYYMMDD_description.sql`.

Axiom 8 — The ORM is the contract, the SQL is the implementation

Direct SQL against the database is allowed in exactly two contexts: the seed scripts (`database/seed_data.py`) and operator-invoked maintenance from the Service Manual (Vol. II, Chapter 18). Production code paths read and write through the ORM so that constraints, enums, and relationship mappings are honoured.

CHAPTER 6

Core Domain Models

The core models are the ones every other table eventually refers to: `User`, `Patient`, `PatientPortalAccount`, and `Insurance`. A reader who understands these four tables has the skeleton for everything that follows.

User — clinical staff

The `User` table holds every staff account. The role column is an enum and governs access control throughout the system.

```python
class UserRole(enum.Enum):
 DOCTOR = "doctor"
 PSYCHIATRIST = "psychiatrist"
 PHARMACIST = "pharmacist"
 NURSE = "nurse"
 ADMIN = "admin"


class User(Base):
 __tablename__ = "users"

 id = Column(Integer, primary_key=True, autoincrement=True)
 username = Column(String(50), unique=True, nullable=False, index=True)
 password_hash = Column(String(256), nullable=False)
 role = Column(Enum(UserRole), nullable=False)
 first_name = Column(String(100), nullable=False)
 last_name = Column(String(100), nullable=False)
 email = Column(String(200), unique=True, nullable=False)
 phone = Column(String(20))
 license_number = Column(String(50))
 npi = Column(String(10), unique=True, nullable=True, index=True)
 specialization = Column(String(200))
 created_at = Column(DateTime, default=datetime.utcnow)
 is_active = Column(Boolean, default=True)

 vitals_recorded = relationship("Vital", back_populates="recorded_by")
 diagnoses_made = relationship("Diagnosis", back_populates="diagnosed_by")
 medical_records = relationship("MedicalRecord", back_populates="provider")
 prescriptions_written = relationship("Prescription", back_populates="prescriber")
 appointments = relationship("Appointment", back_populates="provider")
 audit_logs = relationship("AuditLog", back_populates="user")

 @property
 def full_name(self):
```

```
        return f"{self.first_name} {self.last_name}"

    @property
    def display_title(self):
        if self.role in (UserRole.DOCTOR, UserRole.PSYCHIATRIST):
            return f"Dr. {self.last_name}"
        return self.full_name
```

Listing 6-1. database/models.py — User model (extract).

> **Note.** The `display_title` property is the canonical way to format a provider's name in UI. Any view that writes "Dr. " in front of a user's name on its own is a bug — admins and nurses should not get the title.

Patient — the centre of the domain

The `Patient` table is the hub of nearly every clinical relationship. Its columns are deliberately flat — demographics, a small number of medical flags, emergency contact — with related information pushed out into dedicated tables (Insurance, Allergy, Vital, Diagnosis, and so on).

```
class Patient(Base):
    __tablename__ = "patients"

    id = Column(Integer, primary_key=True, autoincrement=True)
    first_name = Column(String(100), nullable=False)
    last_name = Column(String(100), nullable=False)
    dob = Column(Date, nullable=False)
    gender = Column(Enum(Gender))
    ssn_last4 = Column(String(4))
    email = Column(String(200))
    phone = Column(String(20))
    address = Column(String(300))
    city = Column(String(100))
    state = Column(String(2))
    zip_code = Column(String(10))
    emergency_contact_name = Column(String(200))
    emergency_contact_phone = Column(String(20))
    blood_type = Column(Enum(BloodType))
    created_at = Column(DateTime, default=datetime.utcnow)
    is_active = Column(Boolean, default=True)

    __table_args__ = (
    Index("ix_patients_name", "last_name", "first_name"),
    )
```

Listing 6-2. database/models.py — Patient (demographics).

PatientPortalAccount — separate from Patient

A patient's identity (demographic row) and their portal login credential are deliberately separate rows in separate tables. Not every patient has a portal account — many are elderly, paediatric, or simply

uninterested — and a single portal account is tied to a single patient row. This keeps the demographics table free of authentication concerns and allows portal credentials to be deactivated without affecting the clinical chart.

```
class PatientPortalAccount(Base):
 __tablename__ = "patient_portal_accounts"

 id = Column(Integer, primary_key=True, autoincrement=True)
 patient_id = Column(Integer, ForeignKey("patients.id"),
 unique=True, nullable=False)
 username = Column(String(50), unique=True, nullable=False, index=True)
 password_hash = Column(String(256), nullable=False)
 email_verified = Column(Boolean, default=False)
 created_at = Column(DateTime, default=datetime.utcnow)
 last_login = Column(DateTime)
 is_active = Column(Boolean, default=True)

 patient = relationship("Patient", backref="portal_account")
```

Listing 6-3. Portal account — one-to-one with Patient.

Insurance — multi-row per patient

Patients frequently carry more than one insurance plan (primary and secondary are common; Medicare plus supplemental plans add a third). The `Insurance` table therefore lives in a one-to-many relationship with `Patient`. Each row describes one policy. When a claim is filed, the insurance row is referenced directly; see Chapter 8 for the claim workflow.

```
class CoverageType(enum.Enum):
 PRIMARY = "primary"
 SECONDARY = "secondary"
 TERTIARY = "tertiary"


class Insurance(Base):
 __tablename__ = "insurance"

 id = Column(Integer, primary_key=True, autoincrement=True)
 patient_id = Column(Integer, ForeignKey("patients.id"), nullable=False)
 provider_name = Column(String(200), nullable=False)
 policy_number = Column(String(100), nullable=False)
 group_number = Column(String(100))
 coverage_type = Column(Enum(CoverageType),
 default=CoverageType.PRIMARY)
 copay_amount = Column(Numeric(10, 2), default=0)
 deductible = Column(Numeric(10, 2), default=0)
 effective_date = Column(Date)
 expiry_date = Column(Date)
 is_active = Column(Boolean, default=True)
```

```
22
23  patient = relationship("Patient", back_populates="insurance_records")
```

Listing 6-4. Insurance records belong to a Patient; a patient may have several.

CHAPTER 7

Clinical Models

The clinical models represent the day-to-day output of medical work: vitals observed, diagnoses made, prescriptions written, appointments kept, records filed. Each has its own table; each refers to `Patient` (the subject) and `User` (the clinician responsible).

Medication — the formulary

The `Medication` table is the system's reference formulary. At seed time it holds 75+ drugs with NDC codes, drug classes, DEA schedules, and pricing. It is intentionally static — drugs are not patient-specific. Medication rows are referenced by `PrescriptionItem` and by the drug-interaction table.

```python
class Medication(Base):
 __tablename__ = "medications"

 id = Column(Integer, primary_key=True, autoincrement=True)
 brand_name = Column(String(200), nullable=False)
 generic_name = Column(String(200), nullable=False, index=True)
 ndc_code = Column(String(20), unique=True, index=True)
 drug_class = Column(String(100), index=True)
 dea_schedule = Column(Enum(DrugSchedule), default=DrugSchedule.NONE)
 form = Column(Enum(DrugForm))
 route = Column(Enum(DrugRoute))
 strength = Column(String(100))
 manufacturer = Column(String(200))
 unit_price = Column(Numeric(10, 2), default=0)
 description = Column(Text)
 indications = Column(Text)
 contraindications = Column(Text)
 side_effects = Column(Text)
 is_active = Column(Boolean, default=True)
```

Listing 7-1. Medication — a catalogue row, not a patient-scoped fact.

Prescription and PrescriptionItem — the two-level structure

A prescription is a header (patient, prescriber, status, date, notes, optional `diagnosis_id`) that carries one or more line items (specific medications, dosages, quantities, refills). This two-level structure follows the invoice pattern described in Chapter 8 and makes it natural to add a second medication to an existing script. The `diagnosis_id` foreign key links the prescription to the ICD-10 diagnosis the prescriber selected on the New Prescription dialog (or `NULL` if no diagnosis was attached); the prescriber's `npi` — a 10-digit CMS National Provider Identifier validated against the NPPES Luhn-mod-10 check — is read off the joined `User` row.

```python
class Prescription(Base):
 __tablename__ = "prescriptions"

 id = Column(Integer, primary_key=True, autoincrement=True)
 rx_number = Column(String(30), unique=True, nullable=False, index=True)
 patient_id = Column(Integer, ForeignKey("patients.id"), nullable=False)
 prescriber_id = Column(Integer, ForeignKey("users.id"), nullable=False)
 diagnosis_id = Column(Integer, ForeignKey("diagnoses.id"), nullable=True)
 status = Column(Enum(PrescriptionStatus),
 default=PrescriptionStatus.PENDING)
 prescribed_date = Column(Date, default=date.today)
 expiry_date = Column(Date)
 notes = Column(Text)

 patient = relationship("Patient", back_populates="prescriptions")
 prescriber = relationship("User", back_populates="prescriptions_written")
 diagnosis = relationship("Diagnosis", back_populates="prescriptions")
 items = relationship("PrescriptionItem", back_populates="prescription",
 cascade="all, delete-orphan")


class PrescriptionItem(Base):
 __tablename__ = "prescription_items"

 id = Column(Integer, primary_key=True, autoincrement=True)
 prescription_id = Column(Integer, ForeignKey("prescriptions.id"),
 nullable=False)
 medication_id = Column(Integer, ForeignKey("medications.id"),
 nullable=False)
 dosage = Column(String(100), nullable=False)
 frequency = Column(String(100), nullable=False)
 quantity = Column(Integer, default=0)
 refills_allowed = Column(Integer, default=0)
 refills_used = Column(Integer, default=0)
 instructions = Column(Text)

 prescription = relationship("Prescription", back_populates="items")
 medication = relationship("Medication")
```

Listing 7-2. Prescription two-level structure.

Drug-drug interaction checking

The `MedicationInteraction` table is a many-to-many bridge between two `Medication` rows, annotated with a severity enum and a human-readable description. When a prescription is saved, the `DatabaseManager` checks every pair of items against this table and surfaces any matches to the UI. The table ships with 24 common pairs from the FDA adverse event database; adding new pairs is an ordinary data-maintenance task.

Appointment

Appointments attach to a provider and a patient with a scheduled datetime and duration, and progress through a small state machine: `SCHEDULED` → `CONFIRMED` → `CHECKED_IN` → `IN_PROGRESS` → `COMPLETED`. Alternate exits `CANCELLED` and `NO_SHOW` terminate the state machine early. The Qt Appointments widget and the patient-side mobile clients render the same underlying state machine differently but agree on the transitions.

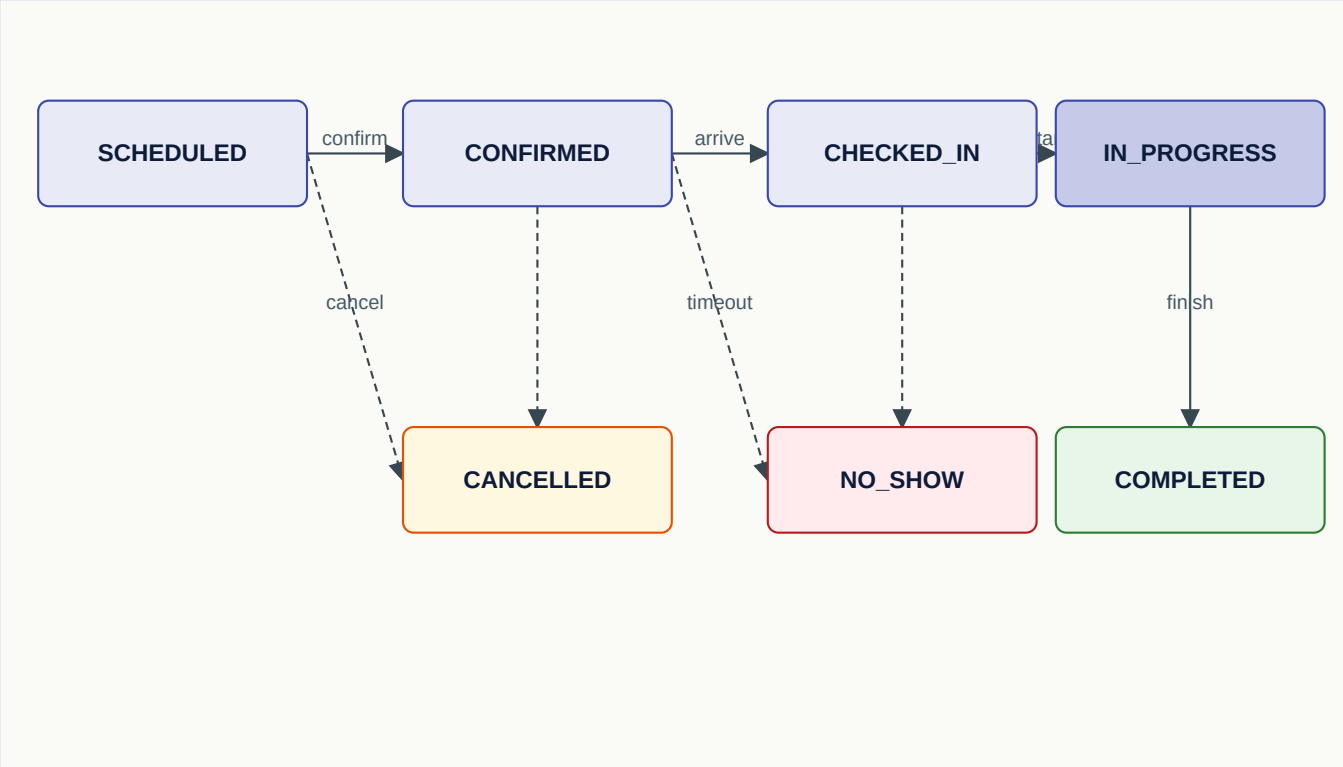

Fig. 7-1. Appointment status transitions. Dashed edges are alternate exits.

MedicalRecord

MedicalRecord is a generic container for clinical documentation — visit notes, lab results, imaging reports, procedure summaries, referrals. The `record_type` enum differentiates them, and the content column holds free text (which may be markdown-formatted). Because this table can hold PHI that the patient is allowed to see (records are rendered in the patient portal), the content is stored as plaintext — encrypting it would make the portal unable to render it without a decryption key on the web side, and that is a larger risk surface than the existing access-control model solves.

Allergy, Vital, Diagnosis

These three tables all hang off `Patient`, are appended to rather than overwritten, and carry a timestamp of when the observation was made and a foreign key to the recording user. They are rendered as history lists in the Qt desktop's patient detail panel and are available through the patient-detail REST endpoint.

Note. Never write over an old Vital row when new vitals are taken. Insert a new row. The history matters.

CHAPTER 8

Financial and Insurance Models

Financial state in MedPharm follows the invoice/payment pattern standard in small-business accounting. Four tables carry the burden: `Invoice`, `InvoiceItem`, `Payment`, and `InsuranceClaim`. Each invoice belongs to a patient; payments and claims both accrete against an invoice; an approved claim produces a payment row automatically.

Invoice and InvoiceItem

The relationship is the same two-level structure used for prescriptions. The header holds the patient, dates, status, and aggregated money fields; the item table holds individual charges with a unit price and quantity.

```
class InvoiceStatus(enum.Enum):
 DRAFT = "draft"
 SENT = "sent"
 PARTIAL = "partial"
 PAID = "paid"
 OVERDUE = "overdue"
 VOID = "void"


class Invoice(Base):
 __tablename__ = "invoices"

 id = Column(Integer, primary_key=True, autoincrement=True)
 invoice_number = Column(String(30), unique=True, nullable=False, index=True)
 patient_id = Column(Integer, ForeignKey("patients.id"), nullable=False)
 prescription_id = Column(Integer, ForeignKey("prescriptions.id"))
 invoice_date = Column(Date, default=date.today)
 due_date = Column(Date)
 total_amount = Column(Numeric(10, 2), default=0)
 amount_paid = Column(Numeric(10, 2), default=0)
 status = Column(Enum(InvoiceStatus), default=InvoiceStatus.DRAFT)
 notes = Column(Text)

 patient = relationship("Patient", back_populates="invoices")
 prescription = relationship("Prescription")
 items = relationship("InvoiceItem", back_populates="invoice",
 cascade="all, delete-orphan")
 payments = relationship("Payment", back_populates="invoice")

 @property
 def balance_due(self):
 return float(self.total_amount or 0) - float(self.amount_paid or 0)
```

Listing 8-1. Invoice header.

Payment

A payment row records money received against an invoice — the method (credit card, debit card, ACH, cash, check), amount, transaction identifier, and status. When a payment posts successfully, the DatabaseManager **updates the parent invoice's** `amount_paid` **and transitions its status** if the balance has reached zero or crossed a threshold. This happens inside the same session as the payment insert — both succeed atomically or both roll back.

InsuranceClaim — the state machine

Claims are a small state machine with real money flowing at the terminal transitions. The flow is: `DRAFT` → `SUBMITTED` → `IN_REVIEW` → (`APPROVED` | `DENIED`) → `PAID`. Claims that are denied may be resubmitted by creating a new draft; the original remains in the audit trail. When a claim transitions to `APPROVED`, the DatabaseManager inserts a `Payment` row for the approved amount and updates the parent invoice in the same transaction.

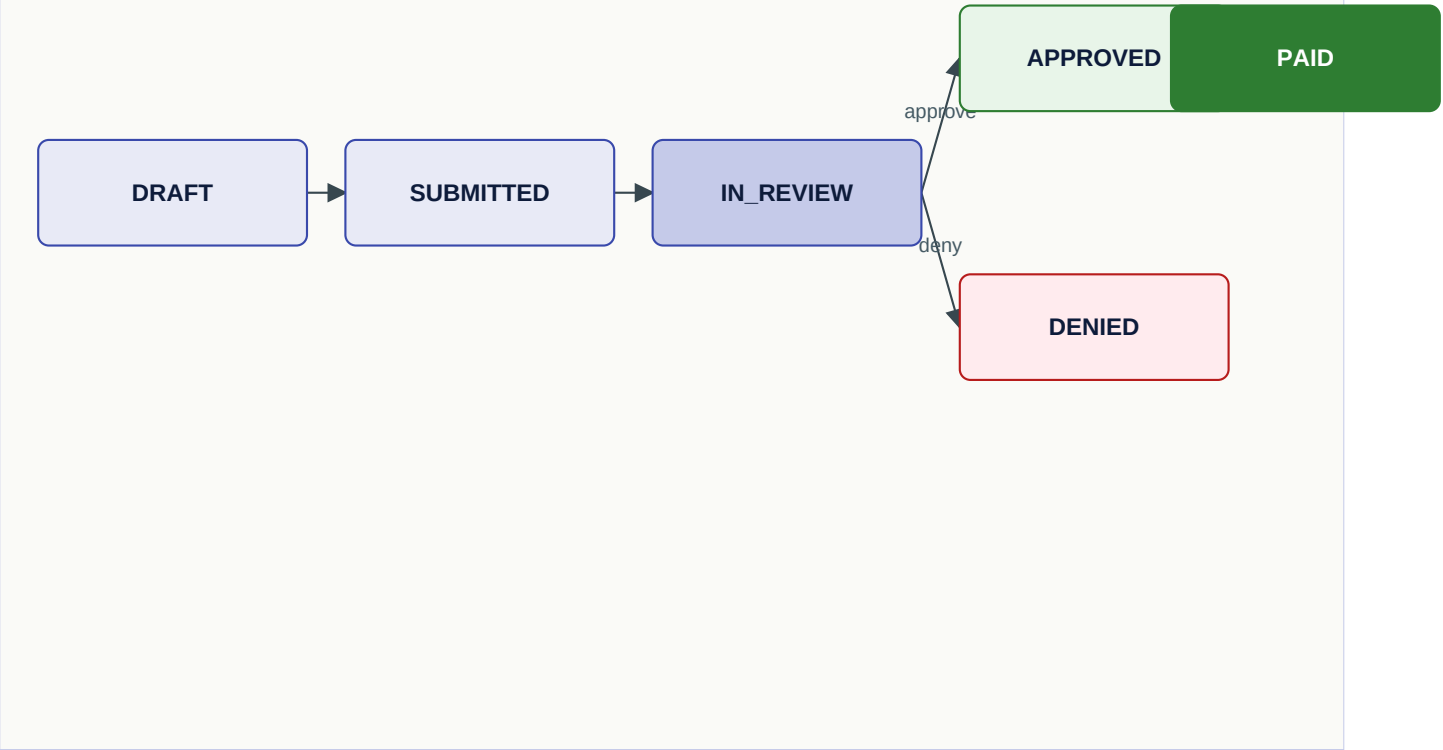

Fig. 8-1. InsuranceClaim lifecycle. Approval triggers a Payment insert.

Money is stored as Numeric, never as float

Every column that holds currency uses SQLAlchemy's `Numeric(10, 2)` type. Floats are forbidden for money. The Python side uses `decimal.Decimal` or `float()` at the display boundary only. This prevents the family of rounding bugs that end with an unexpected cent somewhere.

Caution. If you see `float()` applied to a currency value anywhere in business logic, treat it as a bug. The only legitimate place for float currency is JSON output (because JSON numbers are IEEE 754 anyway), and even there the conversion happens at the last possible moment.

CHAPTER 9

HIPAA Compliance Models

HIPAA's Security Rule requires a documented set of administrative, physical, and technical safeguards. The technical safeguards map to code and therefore to tables. Nine tables exist primarily to satisfy compliance requirements; they share the property that their rows are written constantly and read rarely, except during audit.

AuditLog — who did what

Every action that mutates state — logins, password changes, patient edits, prescription creation, invoice adjustments — is logged to `AuditLog`. The table holds the acting user, timestamp, action name, entity type and id, and a JSON payload of before/after state where relevant. Audit logs are never deleted.

```python
class AuditLog(Base):
 __tablename__ = "audit_logs"

 id = Column(Integer, primary_key=True, autoincrement=True)
 user_id = Column(Integer, ForeignKey("users.id"))
 timestamp = Column(DateTime, default=datetime.utcnow, index=True)
 action = Column(String(100), nullable=False)
 entity_type = Column(String(50))
 entity_id = Column(Integer)
 details_json = Column(Text)
 ip_address = Column(String(45))

 user = relationship("User", back_populates="audit_logs")
```

Listing 9-1. AuditLog — one row per state-changing action.

PHIAccessLog — who looked at what

Where AuditLog records actions that change data, `PHIAccessLog` records actions that merely view it. Opening a patient's chart, fetching a prescription list, rendering the billing history — each produces a row. The schema is similar to AuditLog but the action field is narrower (`VIEW_`-prefixed verbs) and the detail payload is smaller.

FailedLogin and PasswordHistory

`FailedLogin` records every unsuccessful authentication attempt with the account name, timestamp, IP, user agent, and a reason code. The `security/lockout.py` module consults this table when deciding whether to temporarily lock an account. `PasswordHistory` records the last **N** password hashes for each account so that password reuse can be prevented at the policy layer.

MFASecret — TOTP shared secrets

Multi-factor authentication in MedPharm is based on TOTP (RFC 6238). Each user who has enrolled has a row in `MFASecret` with a Fernet-encrypted shared secret. Enrollment, verification, and backup-code handling live in `security/totp.py`.

EmergencyAccessGrantRec — "break the glass"

In an emergency, a clinician may need to access a patient's record outside the normal role-based access controls. The emergency-access mechanism grants a time-boxed, logged, and later-reviewable exception. Each grant is a row in `EmergencyAccessGrantRec` with the user, patient, reason, duration, and approval chain. Every access under the grant produces a PHI access log row, and grants must be reviewed within the period specified by the Security Officer.

PatientConsent — what the patient agreed to

Patients consent to specific uses of their information (treatment, payment, operations, marketing, research, portal access, and so on). Each granted consent is a row in `PatientConsent` with a status, grant and expiry timestamps, a signed signature hash, and a document reference. The `has_active_consent()` method on the DatabaseManager is the authoritative check used by feature code.

```
def has_active_consent(self, patient_id: int, consent_type: str) -> bool:
 with self.get_session() as session:
 row = (session.query(PatientConsent)
 .filter_by(patient_id=patient_id,
 consent_type=ConsentType(consent_type),
 status=ConsentStatus.GRANTED)
 .order_by(desc(PatientConsent.granted_at)).first())
 if not row:
 return False
 if row.expires_at and row.expires_at < datetime.utcnow():
 return False
 return True
```

Listing 9-2. Canonical consent check.

PatientDocument — secure document storage

Consent forms, scanned insurance cards, lab reports, and other file attachments are stored via `PatientDocument` rows, which carry an encrypted filename, a storage reference (path or S3 key), content type, size, an SHA-256 of the ciphertext for integrity, and an uploader. Files themselves are not stored in the database; only the metadata lives there. Chapter 12 describes the encryption of the files on disk.

Summary table

Table	Written on	Retention	Security Rule cite (CFR)
audit_logs	every state change	retained indefinitely	§164.312(b)
phi_access_logs	every PHI read	retained indefinitely	§164.312(b), §164.308(a)(1)(ii)(D)
failed_logins	every bad login	retained at least 6 years	§164.308(a)(5)(ii)(C)
password_histories	every password change	last N per account	§164.308(a)(5)(ii)(D)
mfa_secrets	enrollment	until disabled	§164.312(d)
emergency_access_grants	break-glass	retained indefinitely	§164.312(a)(2)(ii)
patient_consents	consent granted/revoked	retained indefinitely	§164.508
patient_documents	upload	patient-specified	§164.312(c)

CHAPTER 10

Messaging and Private Notes

The messaging and private-notes features are the most recent substantial additions to the schema (version 1.7.5). Both encrypt their payload at rest; both are simple in shape but carry specific access-control rules that are enforced at both the API and DB layers.

MessageThread and SecureMessage

A thread is the conversational container; it belongs to exactly one patient and optionally to one provider. Messages are rows in `SecureMessage`, each tagged with a `sender_type` of either `patient` or `staff`. The body is Fernet-encrypted at rest.

```
class MessageThread(Base):
 __tablename__ = "message_threads"

 id = Column(Integer, primary_key=True, autoincrement=True)
 patient_id = Column(Integer, ForeignKey("patients.id"),
 nullable=False, index=True)
 provider_id = Column(Integer, ForeignKey("users.id"),
 nullable=True, index=True)
 subject = Column(String(200), nullable=False)
 created_at = Column(DateTime, default=datetime.utcnow)
 last_message_at = Column(DateTime, default=datetime.utcnow, index=True)
 is_closed = Column(Boolean, default=False)

 patient = relationship("Patient")
 provider = relationship("User")
 messages = relationship("SecureMessage", back_populates="thread",
 cascade="all, delete-orphan",
 order_by="SecureMessage.sent_at")


class SecureMessage(Base):
 __tablename__ = "secure_messages"

 id = Column(Integer, primary_key=True, autoincrement=True)
 thread_id = Column(Integer, ForeignKey("message_threads.id"),
 nullable=False)
 sender_type = Column(String(20), nullable=False) # staff|patient|system
 sender_id = Column(Integer, nullable=False)
 body_encrypted = Column(Text, nullable=False) # Fernet ciphertext
 sent_at = Column(DateTime, default=datetime.utcnow)
 read_at = Column(DateTime, nullable=True)

 thread = relationship("MessageThread", back_populates="messages")
```

Listing 10-1. MessageThread and SecureMessage.

ProviderNote — author-private

A ProviderNote is a free-text observation written by one clinician and visible to that clinician alone. It exists because the existing `MedicalRecord` container is shared across providers and surfaced to the patient in the portal; those are the wrong semantics for a private working note. The body is Fernet-encrypted at rest, and the API and DB layers both enforce `author_id` equality on every read, update, and delete operation.

```
class ProviderNote(Base):
 __tablename__ = "provider_notes"

 id = Column(Integer, primary_key=True, autoincrement=True)
 patient_id = Column(Integer, ForeignKey("patients.id"),
 nullable=False, index=True)
 author_id = Column(Integer, ForeignKey("users.id"),
 nullable=False, index=True)
 body_encrypted = Column(Text, nullable=False) # Fernet ciphertext
 is_pinned = Column(Boolean, default=False)
 created_at = Column(DateTime, default=datetime.utcnow, index=True)
 updated_at = Column(DateTime, default=datetime.utcnow,
 onupdate=datetime.utcnow)

 patient = relationship("Patient")
 author = relationship("User")
```

Listing 10-2. ProviderNote — visible only to the author.

Danger. Any future feature that surfaces ProviderNote content must verify that the viewing user is the author. The REST routes do this; the Qt desktop does this; no other surface should be added that queries the table without the author filter. If you are tempted to build "share a note with a colleague" — don't. Build a new feature (shared_provider_note) rather than weakening the privacy contract of this one.

Message flow and state

Messages have no internal status beyond their `read_at` timestamp; threads have an `is_closed` flag that prevents further replies. The `mark_messages_read` method sets `read_at` on unread messages sent by the *other side* — staff marking a patient's messages read, or vice versa. It is called automatically when a thread is opened on either client.

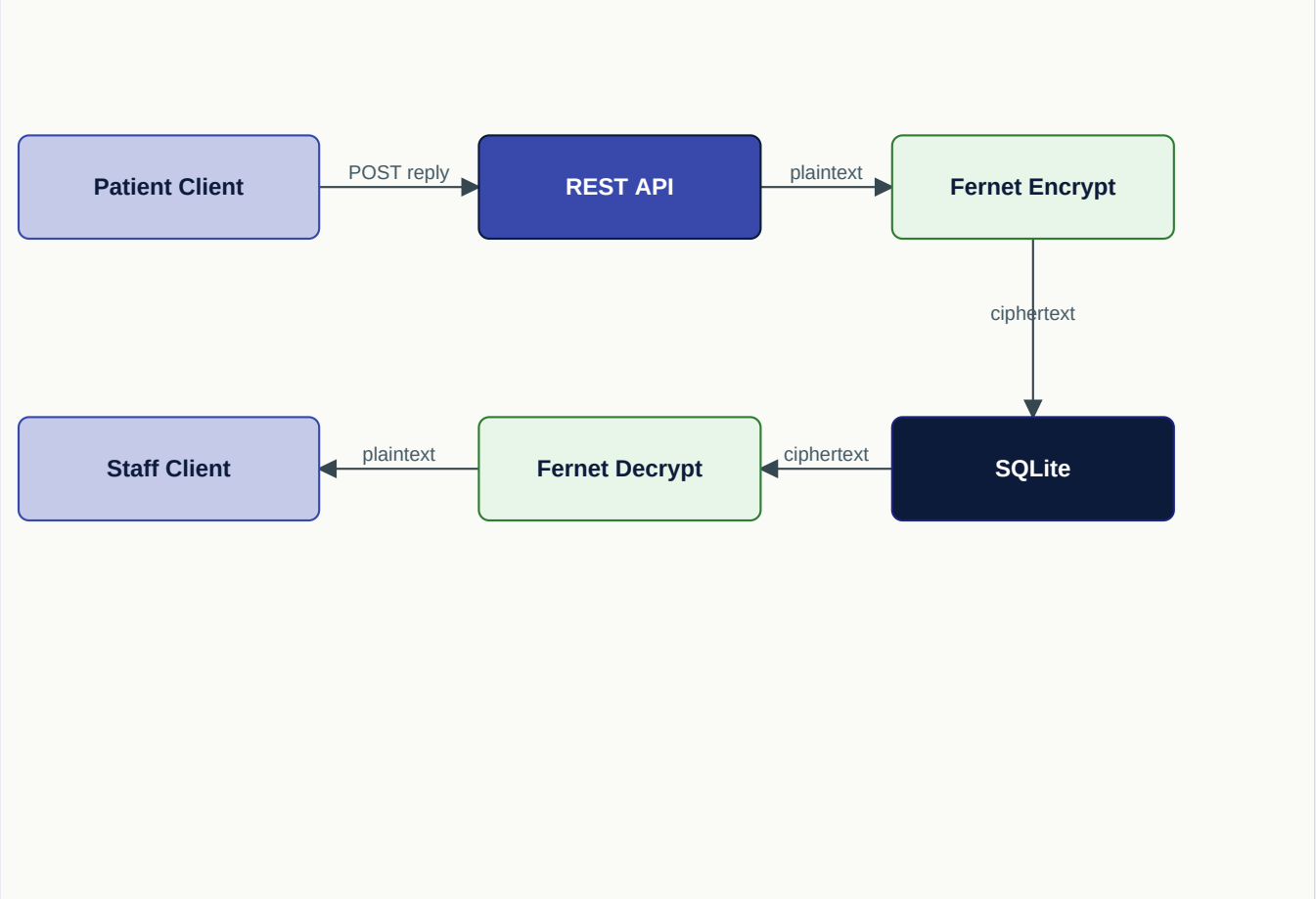

Fig. 10-1. Secure message round-trip. The body is plaintext only inside the Flask process; never on disk.

PART

III

The Backend

From the Flask application factory to the DatabaseManager to the JWT handler — the server-side code paths an engineer touches on a typical day.

CHAPTER 11

The DatabaseManager

The DatabaseManager is the single most important class in the codebase. Every state-changing operation, every query, every business rule that touches more than one row flows through it. This chapter examines its construction in depth.

Instantiation and lifecycle

A DatabaseManager instance is created once per process and given either an explicit `database_url`, a SQLite `db_path`, or neither. Its `init_db()` method resolves the SQLAlchemy URL, creates a dialect-appropriate engine, runs `Base.metadata.create_all`, and stores a session factory for later use. The URL is resolved by a fixed precedence: the explicit `database_url` argument, then the `MEDPHARM_DATABASE_URL` environment variable, then `sqlite:///<db_path>` as the zero-config default. A shared deployment sets `MEDPHARM_DATABASE_URL` so the desktop, portal, API, Docker, and Kubernetes all attach to one database; with nothing set, behaviour is unchanged and each component owns a private SQLite file.

```
class DatabaseManager:
 def __init__(self, db_path: str = None, database_url: str = None):
 self.db_path = db_path
 self._database_url = database_url
 self.engine = None
 self._session_factory = None

 def _resolve_database_url(self):
 # Precedence: explicit arg -> MEDPHARM_DATABASE_URL -> sqlite:///db_path
 url = self._database_url or os.environ.get("MEDPHARM_DATABASE_URL")
 if url:
 return make_url(url)
 if not self.db_path:
 raise ValueError("need a database_url / MEDPHARM_DATABASE_URL "
 "or a SQLite db_path")
 return make_url(f"sqlite:///{self.db_path}")

 def init_db(self):
 url = self._resolve_database_url()
 if url.get_backend_name() == "sqlite":
 # Short-lived sessions handed across threads; the GIL
 # serialises the actual writes, so this stays safe.
 self.engine = create_engine(
 url, echo=False,
 connect_args={"check_same_thread": False},
 )
 else:
```

```
# Networked backend (PostgreSQL): pool_pre_ping discards
# connections the server dropped; pool_recycle caps age.
self.engine = create_engine(
url, echo=False,
pool_pre_ping=True, pool_recycle=1800,
)
Base.metadata.create_all(self.engine)
self._session_factory = sessionmaker(bind=self.engine)

@contextmanager
def get_session(self):
session = self._session_factory()
try:
yield session
session.commit()
except Exception:
session.rollback()
raise
finally:
session.close()
```

Listing 11-1. DatabaseManager core — URL resolution, dialect-aware engine, and session contextmanager.

The `get_session()` contextmanager is the canonical way to perform any DB work. It commits on clean exit and rolls back on exception, so a method body that consists of `with self.get_session() as session:` … is safe even if an SQLAlchemy error occurs inside the `with` block.

Caution. Never create a session manually with `self._session_factory()` outside of `get_session()`. A forgotten close leaks a SQLite file handle — or, on PostgreSQL, holds a pooled connection open; a forgotten rollback leaves the connection in an in-progress transaction.

Method grouping conventions

The DatabaseManager is a large class (over a thousand lines) but its methods are grouped by domain with banner comments that make navigation quick. The groups, in the order they appear, are:

- **Authentication** — password hashing, portal account verification, staff login.
- **Users and portal accounts** — CRUD, provider listing.
- **Patients** — demographics, search, full profile aggregation.
- **Medications and interactions** — catalogue access, interaction checks.
- **Prescriptions** — two-level create, refill, status transitions.
- **Appointments** — calendar queries, status updates.
- **Medical records, vitals, allergies, diagnoses** — append-only history.
- **Invoices and payments** — two-level create, payment posting, balance calculation.

- **Insurance and claims** — policy management, claim state machine, automatic payment on approval.
- **Symptoms and conditions** — reference search with body-system and category filters.
- **Dashboard aggregates** — KPI queries for the home screen of each client.
- **Analytics** — revenue trends, demographics, top medications, provider workload.
- **Security / HIPAA** — audit, PHI access, lockout, password history, MFA, emergency access, consent.
- **Secure messaging** — threads, messages, reader-perspective unread counts.
- **Private provider notes** — encrypted, author-only.
- **Labs and lab results** — ordered tests and results.
- **Care plans and problems** — longitudinal problem lists.
- **Immunizations, referrals, documents** — the rest of the clinical surface area.

A representative method

Most DatabaseManager methods share a common shape: open a session, perform one or more queries, transform to a plain-dict representation suitable for JSON, return. Here is a typical example.

```
def get_message_threads(self, *, patient_id: int | None = None,
 provider_id: int | None = None,
 reader_type: str | None = None) -> list[dict]:
 with self.get_session() as session:
 q = session.query(MessageThread)
 if patient_id is not None:
 q = q.filter(MessageThread.patient_id == patient_id)
 if provider_id is not None:
 q = q.filter(MessageThread.provider_id == provider_id)
 rows = q.order_by(desc(MessageThread.last_message_at)).all()
 out = []
 for t in rows:
 if reader_type == "staff":
 unread = sum(1 for m in t.messages
 if m.sender_type == "patient" and m.read_at is None)
 elif reader_type == "patient":
 unread = sum(1 for m in t.messages
 if m.sender_type == "staff" and m.read_at is None)
 else:
 unread = sum(1 for m in t.messages if m.read_at is None)
 out.append({
 "id": t.id,
 "subject": t.subject,
 "unread_count": unread,
 # … other fields …
 })
```

```
    return out
```

Listing 11-2. A typical DatabaseManager method — filter, order, transform, return.

Eager loading decisions

Relationships default to lazy-loaded; eager loading is opt-in per query. Three patterns are most common:

- **No eager loading** — when the relationship is not accessed in the transform loop. Most queries fall here.
- **joinedload(rel)** — when the relationship is accessed exactly once per row and produces at most a few rows on the other side.
- **selectinload(rel)** — when the relationship is one-to-many with potentially many rows per parent; issues a second `IN` query instead of a potentially huge join.

Tip. If you find yourself debugging a "why is this endpoint slow?" on a patient record, add `echo=True` temporarily to the `create_engine` call and watch the SQL. Eight out of ten such bugs are N+1 loads that want `selectinload`.

Multi-row transactions

Some operations touch multiple tables atomically — creating a prescription also creates an invoice; approving a claim creates a payment and updates the invoice. These are written as a single `with get_session()` block so that they commit together or not at all. The pattern is worth studying before you add similar logic elsewhere.

```
def process_insurance_payment(self, claim_id: int,
 approved: float,
 copay: float,
 deductible: float) -> dict:
 with self.get_session() as session:
 claim = session.get(InsuranceClaim, claim_id)
 if not claim:
 return {"error": "Claim not found"}

 claim.approved_amount = Decimal(str(approved))
 claim.copay_amount = Decimal(str(copay))
 claim.deductible_amount = Decimal(str(deductible))
 claim.status = InsuranceClaimStatus.APPROVED
 claim.processed_at = datetime.utcnow()

 invoice = session.get(Invoice, claim.invoice_id)
 if invoice:
 payment = Payment(
 invoice_id=invoice.id,
 amount=Decimal(str(approved)),
 payment_method=PaymentMethod.INSURANCE,
```

```
    status=PaymentStatus.COMPLETED,
    reference_number=f"CLM-{claim_id}",
    )
    session.add(payment)
    invoice.amount_paid = (invoice.amount_paid or 0) + Decimal(str(approved))
    if invoice.balance_due <= 0:
    invoice.status = InvoiceStatus.PAID
    claim.status = InsuranceClaimStatus.PAID
    elif invoice.amount_paid > 0:
    invoice.status = InvoiceStatus.PARTIAL
    return {"status": "processed", "approved": approved}
```

Listing 11-3. Multi-row transaction — claim, payment, invoice state all update together.

Pagination contract for search_medications

Before the 1.7.6-E medications-catalogue work the medications table held under a hundred rows and the server happily returned every match for any search. After a full FDA NDC bulk load (Phase 1 below) it can hold over 300 000 rows, and an unbounded SELECT against that is guaranteed to bring the API down. `search_medications` is therefore now paginated:

```
def search_medications(self, query="", drug_class="", schedule="", form="",
    limit=_DEFAULT_MED_PAGE_SIZE, # 100
    offset=0,
    include_total=False) -> list[dict] | dict:
    """limit=None means 'give me everything matching' but is silently
    clamped to _UNBOUNDED_MED_LIMIT_CAP (5000) and emits a logged
    WARNING — protect against accidental full-table scans."""
    ...
    if include_total:
    return {"medications": [...], "total": N, "limit": L, "offset": O}
    return [...] # historical signature
```

Listing 11-4. Two response shapes; pick include_total=True when the caller actually paginates.

The legacy `get_all_medications()` is now an alias for `search_medications(limit=100)`. Anyone passing `limit=None` gets clamped to 5000 with a logged warning so a stray unbounded query in a feature branch is loud rather than silent. The Qt MedicationWidget calls the DB on every keystroke (sub-ms with the lower(brand_name) / lower(generic_name) functional indexes added in 1.7.6-E); the Flask route at /medications/search exposes `page` / `page_size` query params and returns `total` + `total_pages` alongside the paginated `medications` list.

Bulk loaders for the medications catalogue

Two scripts populate the table from public US government data. Both are streaming, idempotent, and refuse to overwrite rows whose `data_source` is `'seed'` — the original hand-curated demo data survives any number of bulk loads.

Loader	Source	Provides
load_fda_data.py	FDA NDC Directory (~360k entries)	name, manufacturer, NDC, dose form, route, marketing category, DE
load_fda_data.py	NIH DSLD	complementary / dietary-supplement labels (~150k records)
load_dailymed_spl.py	DailyMed SPL XML labels	indications, contraindications, side effects, drug interactions, dosage

`load_fda_data.py` uses `zipfile + csv.DictReader` for streaming TSV, hint-matches FDA's open-vocabulary dosage form / route strings to the existing `DrugForm` / `DrugRoute` enums, and stores the verbatim FDA strings in the new `dosage_form_raw` / `route_raw` columns so nothing is lost. Upserts batch-commit every 1000 rows to keep memory bounded.

`load_dailymed_spl.py` takes either a list of NDCs (via `--ndc-list` or `--top N`) and fetches per-drug SPL XML from the NLM REST API at 1 req/sec, or a directory of pre-downloaded SPL files via `--from-dir` / `--from-zip` for offline ingest. Per-setid SHA-256 + last-modified is tracked in a `dailymed_ingest_log` side table so a crashed run resumes idempotently. Section extraction uses lxml with `recover=True` for tolerance against malformed SPL exports — that's why `lxml>=5.0.0` is in `requirements-cloud.txt` and baked into the Docker images.

Operator-facing flag matrix and sizing table live at `docs/MEDICATIONS_DATABASE.md`. Rolling back a bad load is one statement: `DELETE FROM medications WHERE data_source IN ('fda_ndc','orange_book','dsld')`.

The _naive_utc_now() helper

Python 3.12 deprecated `datetime.utcnow()`; 3.14 removes it. The repo had 35 call sites — every `Column(DateTime, default=...)` in models.py, various `now = datetime.utcnow()` calls in db_manager.py, and a handful of timestamp formatters in fhir.py / audit.py / emergency.py / seed_data.py. The 1.7.6-D pass replaces them with a single helper:

```python
def _naive_utc_now() -> datetime:
 """Drop-in replacement for the deprecated datetime.utcnow().

 Returns the current UTC time as a *naive* datetime (no tzinfo)
 so it matches the existing schema, where every DateTime column
 has stored naive UTC since the project started. Switching
 wholesale to tz-aware datetimes would require a data migration
 of every historical row.
 """
 return datetime.now(timezone.utc).replace(tzinfo=None)
```

Listing 11-5. database/models.py — the project-wide replacement for datetime.utcnow().

Why naive instead of tz-aware? The existing data on disk is naive UTC; mixing tz-aware values into queries against naive columns triggers `TypeErrors` in SQLAlchemy comparisons. Migrating the schema to tz-aware would require a full row-by-row rewrite of `created_at` / `updated_at` / `audit_log` timestamps, which is more risk than reward for a deprecation that is two Python versions away.

CHAPTER 12

Field-Level Encryption

Certain columns contain sensitive free-text PHI that is never queried and should not sit on disk in plaintext. The `security/encryption.py` module wraps the `cryptography` library's Fernet primitive into a pair of pure-function helpers, `encrypt_field()` and `decrypt_field()`, plus a rotation helper. This chapter walks through the module and describes the key-management lifecycle.

What Fernet is

Fernet is an authenticated encryption scheme defined by the `cryptography` library. Internally it combines AES-128 in CBC mode (for confidentiality) with HMAC-SHA256 (for authentication and integrity) over a timestamp-bearing envelope. Tokens are base64-url-safe encoded. The full specification is at https://github.com/fernet/spec/blob/master/Spec.md; the library API is documented at https://cryptography.io/en/latest/fernet/.

FieldCipher — the singleton

A single `FieldCipher` instance is constructed lazily from the `MEDPHARM_FIELD_KEY` environment variable. The cipher supports a list of legacy keys for rotation — decryption tries each key in order until one succeeds.

```
class FieldCipher:
 def __init__(self, primary_key: str, legacy_keys: list[str] = ()):
 self._primary = Fernet(primary_key.encode())
 self._legacy = [Fernet(k.encode()) for k in legacy_keys]

 def encrypt(self, plaintext: str) -> str:
 return self._primary.encrypt(plaintext.encode("utf-8")).decode("ascii")

 def decrypt(self, ciphertext: str) -> str:
 candidates = [self._primary] + self._legacy
 last_exc = None
 for cipher in candidates:
 try:
 return cipher.decrypt(ciphertext.encode("ascii")).decode("utf-8")
 except InvalidToken as exc:
 last_exc = exc
 raise FieldEncryptionError("No valid key decrypted the field") from last_exc


def encrypt_field(plaintext: str | None) -> str | None:
 if plaintext is None: return None
 return _singleton().encrypt(plaintext)

```

```
24
25 def decrypt_field(ciphertext: str | None) -> str | None:
26  if ciphertext is None: return None
27  return _singleton().decrypt(ciphertext)
```

Listing 12-1. FieldCipher — multi-key-aware wrapper over Fernet.

Which columns are encrypted

Table	Column	Why
secure_messages	body_encrypted	Free-form clinical communication, never queried by substring.
provider_notes	body_encrypted	Author-private working notes; never queried by text.
mfa_secrets	secret	TOTP shared secret; leakage would compromise the second factor.
patient_documents	filename_encrypted	Original filenames may contain PHI ("smith-john-mri-2024.pdf").

The rotation workflow

Keys are rotated according to the policy described in the Service Manual, Chapter 15. The engineering steps are:

- Generate a new key with `generate_key()` and set it as `MEDPHARM_FIELD_KEY` in the new environment.
- Move the previous key into `MEDPHARM_FIELD_KEY_LEGACY` (comma-separated for multiple legacy keys).
- Restart the application. New writes are encrypted with the primary key; existing rows are decrypted under the legacy key on read.
- Over time, a background re-encryption pass may rewrite existing rows with the new key and retire the legacy key. This is an operational decision, not an architectural requirement; dual-key support can be kept indefinitely.

Danger. If all legacy keys are removed before existing ciphertext has been re-encrypted, the existing rows become **permanently unreadable**. There is no backdoor. Always verify by sampling a handful of rows before dropping a legacy key.

Why not column-level transparent encryption?

SQLCipher and similar transparent-database-encryption schemes would be simpler in some respects, but they encrypt the whole database under a single key. MedPharm's field-level approach lets us keep queryable columns (patient name, dates) indexable and searchable while protecting the fields that do not need to be either. It also lets the key rotation happen online without re-opening the database file.

CHAPTER 13

REST API Design

The REST API is a Flask blueprint registered under `/api/v1/`. The prefix is deliberately versioned so that future evolutions of the protocol can cohabit with older clients. This chapter describes the endpoint conventions, the request/response shapes, and the common idioms you will reproduce when adding a new route.

URL conventions

- All URLs are lower-case, words separated by hyphens where necessary.
- Collections are plural: `/patients`, `/prescriptions`.
- Items within collections are addressed by integer id: `/patients/42`, `/prescriptions/17/items`.
- Actions that do not fit REST semantics are expressed as verbs under the owning resource: `/staff/messages/17/reply`, `/staff/messages/17/close`.
- Patient-facing endpoints live under `/patient/`. Staff-facing endpoints live under `/staff/`. Reference data (medications, symptoms, conditions) lives at the top level.

Response conventions

- Success responses always return JSON with a 2xx status.
- The top-level object has a single key whose name matches the resource — e.g. `{"threads": [...]}` — or a small number of summary keys for dashboard-style endpoints.
- Errors return a 4xx or 5xx status with a body shaped `{"error": ""}`. Clients should rely on the status code for programmatic handling and the message only for user display.
- Creation endpoints return the new resource's id and 201, not the full resource. The client must re-read if it needs the complete shape.

A complete route example

```
@api_bp.route("/patient/messages/<int:thread_id>/reply", methods=["POST"])
@patient_required
def patient_reply_thread(thread_id):
 thread = g.db_manager.get_message_thread(thread_id)
 if not thread or thread["patient_id"] != g.current_patient_id:
 return jsonify({"error": "Thread not found"}), 404
 if thread["is_closed"]:
 return jsonify({"error": "Thread is closed"}), 400
 data = request.get_json(silent=True) or {}
 body = (data.get("body") or "").strip()
```

```
if not body:
return jsonify({"error": "Body required"}), 400
msg_id = g.db_manager.post_secure_message(
thread_id=thread_id, sender_type="patient",
sender_id=g.current_user_id, body_plain=body)
return jsonify({"message_id": msg_id}), 201
```

Listing 13-1. A canonical route — decorator, validation, DB call, return.

Observe the three phases. **Validate**: the thread exists, it belongs to this patient, it is not closed, the body is non-empty. **Perform**: a single DatabaseManager call. **Return**: the new id plus 201. Routes longer than about twenty lines are a code smell — the excess usually wants to move into the DatabaseManager.

CORS and versioning

CORS is enabled via `flask-cors` with origins configured by the `MEDPHARM_CORS_ORIGINS` environment variable (default `*` for development; specific origins for production). The single `v1` prefix is the only version in existence at present; when a breaking change is required, add a `v2` blueprint alongside and keep `v1` working for the lifetime of any client that depends on it.

Endpoint catalogue

A complete table of every endpoint — method, path, required auth, request body, response shape — appears in Appendix C. When adding a new endpoint, update Appendix C in the same pull request.

CHAPTER 14

JWT Authentication Flow

MedPharm's JWT implementation is deliberately hand-written in `api/auth.py`. It uses HMAC-SHA256 over a JSON payload — the `hmac` and `hashlib` standard library modules do the entire job, and no third-party JWT library is involved. This chapter explains why, walks through the token lifecycle, and describes the three access-control decorators.

Why not PyJWT or python-jose

Third-party JWT libraries have historically shipped a non-trivial number of verification-bypass vulnerabilities (algorithm confusion, `alg=none` acceptance, key-type confusion, etc.). The MedPharm token format is narrow: HMAC-SHA256 is the only supported algorithm, tokens have a fixed header, and verification uses `hmac.compare_digest` to avoid timing attacks. Hand-writing the forty lines of code that suffice closes the door on those vulnerability classes.

Token format

A MedPharm token is two base64url-encoded segments separated by a dot: `.`. The payload is a small JSON object; the signature is HMAC-SHA256 over the payload segment using `MEDPHARM_JWT_SECRET`. Payload fields:

Field	Type	Meaning
user_type	str	"staff" or "patient"
user_id	int	User or portal account id
patient_id	int	Associated patient id (patient tokens only)
username	str	Account username
role	str	Staff role (doctor, admin, …)
name	str	Display name
iat	int	Issued-at (unix seconds)
exp	int	Expiry (unix seconds)
typ	str	"access" or "refresh"

The implementation

```
1 def _b64url_encode(data: bytes) -> str:
```

```
 return base64.urlsafe_b64encode(data).rstrip(b"=").decode("ascii")


def _sign(payload_b64: str) -> str:
 signature = hmac.new(
 _SECRET_KEY.encode("utf-8"),
 payload_b64.encode("utf-8"),
 hashlib.sha256,
 ).digest()
 return _b64url_encode(signature)


def create_token(user_type, user_id, patient_id=None, username="",
 role="", name="", is_refresh=False):
 expiry = _REFRESH_EXPIRY_SECONDS if is_refresh else _TOKEN_EXPIRY_SECONDS
 payload = {
 "user_type": user_type, "user_id": user_id,
 "patient_id": patient_id, "username": username,
 "role": role, "name": name,
 "iat": int(time.time()),
 "exp": int(time.time()) + expiry,
 "typ": "refresh" if is_refresh else "access",
 }
 payload_json = json.dumps(payload, separators=(",", ":"))
 payload_b64 = _b64url_encode(payload_json.encode("utf-8"))
 sig = _sign(payload_b64)
 return f"{payload_b64}.{sig}"


def decode_token(token: str) -> dict | None:
 try:
 parts = token.split(".")
 if len(parts) != 2:
 return None
 payload_b64, sig = parts
 expected_sig = _sign(payload_b64)
 if not hmac.compare_digest(sig, expected_sig):
 return None
 payload_json = _b64url_decode(payload_b64).decode("utf-8")
 payload = json.loads(payload_json)
 if payload.get("exp", 0) < time.time():
 return None
 return payload
 except Exception:
 return None
```

Listing 14-1. The whole JWT implementation.

Authentication sequence

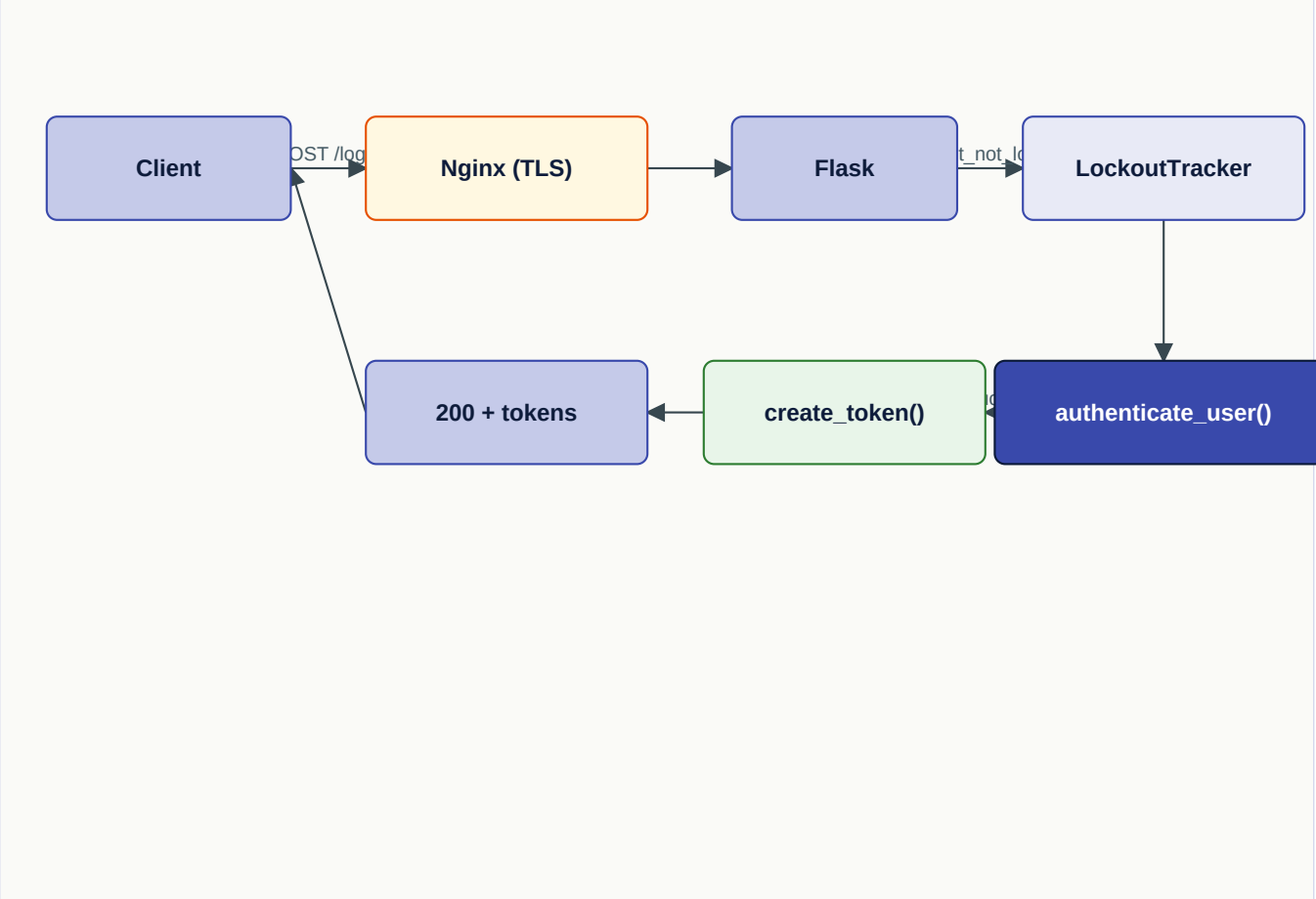

Fig. 14-1. Login sequence. Lockout check happens before password verification.

The three decorators

```python
def token_required(f):
 @wraps(f)
 def wrapper(*args, **kwargs):
 auth_header = request.headers.get("Authorization", "")
 if not auth_header.startswith("Bearer "):
 return jsonify({"error": "Missing or invalid Authorization header"}), 401
 token = auth_header[7:]
 payload = decode_token(token)
 if not payload:
 return jsonify({"error": "Invalid or expired token"}), 401
 if payload.get("typ") != "access":
 return jsonify({"error": "Access token required"}), 401
 g.token_payload = payload
 g.current_user_type = payload["user_type"]
 g.current_user_id = payload["user_id"]
 g.current_patient_id = payload.get("patient_id")
```

```
 g.current_role = payload.get("role", "")
 return f(*args, **kwargs)
 return wrapper


def patient_required(f):
 @wraps(f)
 @token_required
 def wrapper(*args, **kwargs):
 if g.current_user_type != "patient":
 return jsonify({"error": "Patient access required"}), 403
 return f(*args, **kwargs)
 return wrapper


def staff_required(f):
 @wraps(f)
 @token_required
 def wrapper(*args, **kwargs):
 if g.current_user_type != "staff":
 return jsonify({"error": "Staff access required"}), 403
 return f(*args, **kwargs)
 return wrapper
```

Listing 14-2. Access-control decorators.

Three rules cover almost all routes. `@patient_required` — endpoints a patient hits from their mobile or web client, scoped to their own data. `@staff_required` — endpoints a clinician or admin hits, scoped by role only when further restriction is required inside the handler. `@token_required` — endpoints that accept either side, like the medication reference search.

CHAPTER 15

CSRF, Lockout, and MFA

Three security modules sit between the application factory and the routes: `security/csrf.py` for Cross-Site Request Forgery protection on the web portal, `security/lockout.py` for brute-force-resistant login, and `security/totp.py` for multi-factor authentication. Each is worth understanding before you add a feature that touches authentication.

CSRF

The REST API does not need CSRF protection — it uses bearer tokens, not cookies, and an attacker's site cannot induce a browser to attach someone else's Bearer header. The Flask web portal, however, uses session cookies and must be protected. The `@csrf_required` decorator applies to any portal route that mutates state. The token is generated per session, included in every form, and verified on POST.

```
def csrf_required(f):
 @wraps(f)
 def wrapper(*args, **kwargs):
 if request.method in ("POST", "PUT", "DELETE", "PATCH"):
 token_form = request.form.get("csrf_token") or request.headers.get("X-CSRF-Token")
 token_session = session.get("csrf_token")
 if not token_form or not token_session or not hmac.compare_digest(token_form, token_session):
 abort(403, description="CSRF verification failed")
 return f(*args, **kwargs)
 return wrapper
```

Listing 15-1. CSRF decorator — session-stored token compared with constant-time equality.

Lockout

The `LockoutTracker` in `security/lockout.py` keeps an in-memory map of *key* → *(failure count, first-failure-time, locked-until)*. Keys are typically `staff:username.lower()` or `api-patient:username.lower()`. A configurable threshold triggers a lockout for a configurable duration, after which the counter is reset.

```
class LockoutTracker:
 def __init__(self, threshold=5, window_seconds=300, lockout_seconds=900):
 self.threshold = threshold
 self.window = window_seconds
 self.lockout = lockout_seconds
 self._state: dict[str, tuple[int, float, float]] = {}

 def assert_not_locked(self, key):
```

```
now = time.time()
state = self._state.get(key)
if state and state[2] > now:
raise AccountLockedError(retry_after=int(state[2] - now))

def record_failure(self, key):
now = time.time()
count, first, _ = self._state.get(key, (0, now, 0))
if now - first > self.window:
count = 0
first = now
count += 1
locked_until = now + self.lockout if count >= self.threshold else 0
self._state[key] = (count, first, locked_until)

def record_success(self, key):
self._state.pop(key, None)
```

Listing 15-2. LockoutTracker — simple sliding-window counter with cooldown.

> **Note.** The tracker is process-local. Under gunicorn with multiple workers, an attacker's attempts are spread across workers; the threshold is therefore effectively multiplied by the worker count. For deployments with significant concurrency, consider pinning the tracker to a single worker or externalising it to Redis. At current scale, process-local is sufficient.

MFA — TOTP

`security/totp.py` implements TOTP (RFC 6238) with a 30-second step and SHA-1 HMAC. The shared secret is generated at enrollment, encoded as a `otpauth://` URL presentable as a QR code, and stored Fernet-encrypted in the `MFASecret` table. Verification accepts the current step ±1 (so a one-step clock skew is tolerated) and produces a single-use backup code set on initial enrollment.

The MFA flow is optional at present — a site may run with MFA disabled — but it is recommended for any deployment that handles real PHI. See the Service Manual (Vol. II, Chapter 10) for operator-facing enrollment procedures.

CHAPTER 16

The Flask Web Portal

The web portal is a classic server-rendered Flask app living under `web/`. It serves patients who prefer a browser to an app — elderly patients, patients on a Chromebook, patients on a desktop computer at work. It uses Flask-session session cookies rather than JWT because the browser round-trip is naturally cookie-based.

Application factory

```
def create_app(db_manager):
 app = Flask(__name__, static_folder="static", template_folder="templates")
 app.config["SECRET_KEY"] = os.environ.get("MEDPHARM_SECRET_KEY", _dev_secret())
 app.config["SESSION_COOKIE_HTTPONLY"] = True
 app.config["SESSION_COOKIE_SAMESITE"] = "Lax"
 app.config["SESSION_COOKIE_SECURE"] = (os.environ.get("MEDPHARM_TLS_MODE") != "disable")

 @app.before_request
 def attach_db():
 g.db_manager = db_manager

 @app.context_processor
 def inject_globals():
 return {
 "current_year": datetime.utcnow().year,
 "user_logged_in": "user_id" in session,
 "patient_name": session.get("patient_name", ""),
 "csrf_token": _ensure_csrf_token(),
 }

 app.register_blueprint(portal_bp)
 return app
```

Listing 16-1. Web portal application factory.

Routing conventions

- Every mutating route is decorated with `@login_required` and `@csrf_required`.
- Routes return rendered Jinja2 templates, not JSON.
- On success, mutating routes redirect with `redirect(url_for(...))` rather than re-rendering — the POST/Redirect/GET pattern prevents double-submit on refresh.
- Flash messages are used for user-visible confirmations and errors. The base template renders them in a Bootstrap alert.

Templates

Templates live in `web/templates/` and extend `base.html`. The base template carries the top navigation bar, notification bell, logout menu, and footer. Each functional area (prescriptions, billing, records, messages, appointments, medications, profile) has its own template pair — a list view and, where applicable, a detail view and a form.

The CSS is a hand-written dark-gradient theme in `static/css/style.css` with Bootstrap 5 as the grid and component layer. Font Awesome is vendored under `static/vendor/fontawesome/`, Bootstrap under `static/vendor/bootstrap/`, and Google Fonts cached locally — no CDN is contacted at runtime so that the portal works in air-gapped deployments.

Patient authentication flow

The portal's login flow is simpler than the JWT flow. On POST to `/login`, the handler calls `db_manager.authenticate_portal(username, password)`. On success, the user_id, patient_id, and display name are written to the Flask session; on failure, the LockoutTracker records the attempt. Registration uses four-factor identity verification (name, DOB, SSN last 4, insurance id) against an existing Patient row.

In-portal User Guide

The portal ships its own User Guide at `web/templates/help.html`, a 17-section patient-focused walkthrough that mirrors the desktop client's bundled `qt_app/resources/help.html`. The route is `GET /help` (handler `portal.help_page` in `web/routes.py`). It is intentionally *not* decorated with `@login_required` so the URL can be shared with patients before their first sign-in. The link sits in the top-right user dropdown rendered by `base.html`, between Profile and Logout.

```
@portal_bp.route("/help")
def help_page():
 """In-portal user guide. Linked from the right-hand user dropdown."""
 return render_template("help.html")
```

Listing 16-2. /help route — no auth, returns the bundled guide.

The template extends `base.html` so the navbar stays in place, then injects its own scoped stylesheet that borrows the portal's design tokens (`--mp-primary`, `--mp-surface`, `--mp-border`, and the gradient set). A sticky left-rail table of contents tracks the active section; on screens narrower than 992 px the rail collapses inline above the content. When you add a new portal screen, add a matching numbered section to the guide and a link in the TOC — the file is checked into source control alongside the feature, so the docs cannot drift away from the code.

PART

IV

The Clients

Five client surfaces share one backend. Each chapter describes the idioms, the code layout, and the conventions to follow when extending that client.

CHAPTER 17

Qt6 Desktop Application

The clinical desktop application is the most feature-rich client surface. It is a PyQt6 program that embeds the DatabaseManager in-process and drives a dark-themed sidebar-navigation window with one widget per functional area. This chapter walks through the major files and explains how a new widget is added.

Entry point and main window

`run_qt.py` creates a `QApplication`, constructs a `DatabaseManager`, calls `init_db()`, and passes both to the `MainWindow` constructor. By default that manager opens a local SQLite file; if `MEDPHARM_DATABASE_URL` is set in the desktop's environment, the very same construction attaches it to the shared PostgreSQL backend instead, with no other code change. The main window first shows a `LoginDialog`; on success it sets up the sidebar, the `QStackedWidget`, and the status bar.

```
class MainWindow(QMainWindow):
 NAV_ITEMS = [
 ("Dashboard", "dashboard"),
 ("Patients", "patients"),
 ("Prescriptions", "prescriptions"),
 ("Medications", "medications"),
 ("Appointments", "appointments"),
 ("Records", "records"),
 ("Billing", "billing"),
 ("Messages", "messages"),
 ("Symptoms", "symptoms"),
 ("Analytics", "analytics"),
 ]

 def __init__(self, db_manager):
 super().__init__()
 self.db_manager = db_manager
 self.current_user = None
 self.setWindowTitle("MedPharm ERP")
 self.setMinimumSize(1400, 900)
 self.setStyleSheet(STYLESHEET)

 if not self.do_login():
 return

 self.setup_ui()
 self.setup_menu()
 self.setup_statusbar()
 self.navigate_to(0)
```

Listing 17-1. MainWindow constructor and nav item list.

Widget responsibilities

Each widget lives in its own file under `qt_app/widgets/` and follows a standard shape: a `__init__` that calls `setup_ui`, a `refresh_data` method that is invoked on navigation, and a set of event handlers that call methods on `self.db_manager`. The widget **never** builds SQL or manipulates ORM objects directly — it talks only through the DatabaseManager.

Dialog idioms

Dialogs (`QDialog`) are used for modal flows: new patient, new prescription, payment, insurance claim submission, provider note editor. The convention is: build the form, wire Cancel to `dialog.reject()`, wire Save to a method on the parent widget that validates and calls the DatabaseManager. On success, the parent widget refreshes its display.

Stylesheet

`qt_app/styles.py` holds a QSS stylesheet applied globally at startup. The palette is teal-on-dark with a few semantic accents — pinned notes use amber; critical allergy warnings use crimson. Object names are set consistently (`setObjectName("heading")`, `"primary_button"`, `"card"`, `"nav_button"`) so the stylesheet selectors can target them.

Threading

Long-running database work is rare in clinical widgets — SQLite on the same machine answers most queries in a millisecond. A desktop pointed at a shared PostgreSQL backend over the network pays a round-trip per query instead, so this assumption weakens; the analytics widget, which computes aggregates for matplotlib charts, is the first place that shows. It runs its queries synchronously on the main thread today, and if that becomes painful — especially against a remote database — it will move to a `QThread`. In general, prefer to keep the event loop responsive and only introduce threads when profiling demands it.

Adding a new widget

Recipe (expanded in Chapter 33): create `qt_app/widgets/yourname_widget.py` with a class `YourNameWidget(QWidget)`, import it in `main_window.py`, add a tuple to `NAV_ITEMS`, add the instantiation to the stack, done.

Bundled User Guide

The desktop ships a self-contained 23-section User Guide at `qt_app/resources/help.html`, reached from **Help** → **User Guide** (**F1**). The `Help` menu is built in `MainWindow.setup_menu`; the action is wired to `show_user_guide`, which delegates to

`QDesktopServices.openUrl(QUrl.fromLocalFile(...))` so the operator's default browser handles rendering. If the bundled file is missing — for example, a partial install — the handler falls back to a `QMessageBox` warning rather than failing silently. The web portal carries an analogous `/help` route documented in Chapter 16; both files are versioned alongside the code that they describe.

CHAPTER 18

Android (Kotlin / MVVM / Retrofit)

The Android application is a patient-facing client built on Jetpack's modern stack. This chapter walks through the structure, the concurrency model, the secure-storage strategy, and the conventions you should follow when adding a new screen.

Project layout

- `android/` — the Gradle project root; contains `build.gradle.kts`, `settings.gradle.kts`, and the `app/` module.
- `app/src/main/java/com/enlightec/medpharm/` — Kotlin sources; see subdirectories below.
- `data/api/` — `ApiService` interface, `ApiClient` builder, `ServerConfig`.
- `data/model/` — all `data class`es that map to JSON.
- `data/repository/` — `MedPharmRepository`, the central facade over Retrofit calls.
- `ui//` — one subdirectory per feature; each contains Fragment, ViewModel, Adapter, and any feature-specific Activities.
- `util/` — `TokenManager`, `AuthInterceptor`, `Resource` sealed result type.

The Resource sealed class

```
sealed class Resource<out T> {
 object Loading : Resource<Nothing>()
 data class Success<out T>(val data: T) : Resource<T>()
 data class Error(val message: String, val code: Int = 0) : Resource<Nothing>()
}
```

Listing 18-1. The Resource wrapper — single return type for any async repository call.

Every repository method returns `Resource`. ViewModels expose `LiveData>`. Fragments observe it and switch on the three cases — Loading, Success, Error. This uniform shape removes a class of nullability bugs and makes UI state handling boring in the best sense.

Retrofit and ApiService

The `ApiService` interface enumerates every endpoint with Retrofit annotations. Adding a new endpoint is a two-line change here plus one method on the repository.

```
interface ApiService {
 @POST("api/v1/auth/login/patient")
 suspend fun patientLogin(@Body body: Map<String, String>):
Response<LoginResponse>
```

```
4
5  @GET("api/v1/patient/messages")
6  suspend fun getMessageThreads(): Response<MessageThreadsResponse>
7
8  @POST("api/v1/patient/messages/{threadId}/reply")
9  suspend fun replyToThread(
10 @Path("threadId") threadId: Int,
11 @Body body: ReplyRequest,
12 ): Response<MessageResponse>
13 }
```

Listing 18-2. ApiService excerpt — declarative endpoint bindings.

Auth interceptor and token storage

Tokens are stored in `EncryptedSharedPreferences` through the `TokenManager` singleton. The `AuthInterceptor` runs on every OkHttp request, adds the `Authorization: Bearer` header, and on 401 triggers a refresh attempt. If refresh fails, the user is bounced to `LoginActivity`.

Network security configuration

`res/xml/network_security_config.xml` pins the runtime to TLS-only connections to the production hostname, with a for `localhost` and `10.0.2.2` (the Android emulator's alias for the host machine) that trusts user-installed CAs so developers can accept the MedPharm self-signed cert.

Adding a new screen

The pattern is: data class → ApiService method → repository method → ViewModel → Fragment + layout. The Messages feature, added in 1.7.5, is a complete example (`ui/messages/`) — use it as a template.

CHAPTER 19

iOS (SwiftUI / async-await)

The iOS application is a SwiftUI app targeting iOS 16+. It uses async/await for all networking, NavigationStack for hierarchical navigation, and the iOS Keychain for token storage. The code is deliberately short — SwiftUI declarative syntax and a small surface area keep the file count low.

Project layout

- `ios/MedPharm/` — Xcode project root.
- `MedPharm/MedPharmApp.swift` — `@main` entry point, conditional root view based on auth state.
- `MedPharm/Models/Models.swift` — all `Codable` data types.
- `MedPharm/Services/APIClient.swift` — actor-based HTTP client with automatic token refresh.
- `MedPharm/Services/AuthManager.swift` — login state management as an `ObservableObject`.
- `MedPharm/Views//` — one directory per feature area.
- `MedPharm/Theme.swift` — `MPColor` enum and a reusable gradient background view.

The APIClient actor

Swift's `actor` keyword is perfect for a network client with mutable shared state (tokens). The APIClient is an actor whose methods can be called concurrently but whose internal state is serialised automatically.

```
actor APIClient {
 static let shared = APIClient()
 private var baseURL: String = APIClient.storedBaseURL
 private var accessToken: String? {
 get { KeychainHelper.get(key: "access_token") }
 set { KeychainHelper.set(key: "access_token", value: newValue) }
 }

 func request<T: Decodable>(_ endpoint: String, method: String = "GET",
 body: Encodable? = nil) async throws -> T {
 guard let url = URL(string: "\(baseURL)/\(endpoint)") else {
 throw APIError.invalidURL
 }

 var request = URLRequest(url: url)
 request.httpMethod = method
```

```
request.setValue("application/json", forHTTPHeaderField: "Content-Type")
if let token = accessToken {
request.setValue("Bearer \(token)", forHTTPHeaderField: "Authorization")
}
if let body = body {
request.httpBody = try JSONEncoder().encode(body)
}

let (data, response) = try await URLSession.shared.data(for: request)
guard let httpResponse = response as? HTTPURLResponse else {
throw APIError.invalidResponse
}
if httpResponse.statusCode == 401 {
if let newToken = try? await refreshAccessToken() {
accessToken = newToken
// retry once
request.setValue("Bearer \(newToken)", forHTTPHeaderField: "Authorization")
let (retryData, _) = try await URLSession.shared.data(for: request)
return try JSONDecoder().decode(T.self, from: retryData)
}
throw APIError.unauthorized
}
guard (200...299).contains(httpResponse.statusCode) else {
throw APIError.serverError(httpResponse.statusCode)
}
return try JSONDecoder().decode(T.self, from: data)
}
}
```

Listing 19-1. APIClient actor — single generic entry point with automatic token refresh.

View composition

Views follow the conventional SwiftUI pattern: a `@State` property for each user-edited field, an `@StateObject` or `@EnvironmentObject` for any shared service (primarily `AuthManager`), and `.task` modifiers to perform initial loads. The `MessagesView`, added in 1.7.5, is a good compact example of the full pattern — list → detail → compose.

Keychain storage

Tokens live in the iOS Keychain via the `KeychainHelper` utility. The Keychain persists across app uninstalls by default; the app explicitly clears entries on logout. The server-URL override lives in `UserDefaults` — it is operational configuration, not a secret, and is more conveniently inspected in UserDefaults for support purposes.

CHAPTER 20

macOS (SwiftUI / NavigationSplitView)

The macOS client shares most of its code with the iOS client. The Models, APIClient, and AuthManager are essentially identical; the Views differ where macOS idioms (NavigationSplitView, contextual menus, table columns) warrant different layout.

What is different from iOS

- The root view is a `NavigationSplitView` with a permanent sidebar of destinations, replacing iOS's tab bar.
- Lists use `List` with selection binding rather than `NavigationLink` rows — this gives the canonical Mac master-detail feel.
- Dialogs are `sheet`s with explicit frame sizes, because Mac modals expect a reasonable minimum size.
- The Messages view uses an inner `HSplitView` so that the thread list and the conversation pane are independently resizable.
- There is no `navigationBarTitleDisplayMode`; toolbar items go in `.primaryAction` or `.automatic` placements instead of iOS's topBarLeading/topBarTrailing.

Shared codebase

Where the Models, Services, or Views are materially similar, the macOS files are near-copies of the iOS files with the comment header adjusted. This is a deliberate trade-off: a shared Swift package would reduce duplication but complicate the Xcode project graph. With two targets only, the maintenance cost is small — a new Model added on iOS is a copy-paste to macOS.

Note. If you are adding a model field, update *both* `ios/.../Models.swift` and `macos/.../Models.swift` in the same commit. A drift between them is a class of subtle bug that has happened before.

Building and packaging

Open `macos/MedPharm/MedPharm.xcodeproj` in Xcode and build. For TestFlight/App Store distribution, the usual signing and provisioning steps apply; for internal distribution of unsigned builds, the Service Manual (Vol. II, Ch. 24) documents the deployment path.

CHAPTER 21

Windows (.NET 8 / WPF)

The Windows client is a WPF application on .NET 8 using the MVVM pattern via the CommunityToolkit.Mvvm package. It exists because many medical practices run Windows at the front desk and benefit from a native desktop client rather than a browser.

Project layout

- `windows/MedPharm/` — solution root.
- `MedPharm.csproj` — project file (`true`, `net8.0-windows`).
- `App.xaml` / `App.xaml.cs` — application startup, theme loading, global `ApiClient` and `TokenStore` singletons.
- `Models/ApiModels.cs` — POCOs matching the JSON shapes.
- `Services/ApiClient.cs` — `HttpClient`-based API client with `Newtonsoft.Json` serialisation.
- `Services/TokenStore.cs` — DPAPI-encrypted token storage (`ProtectedData` with `DataProtectionScope.CurrentUser`).
- `Views/` — `MainWindow.xaml` plus `Pages/` (UserControls) for each navigation destination.
- `Themes/MedPharmTheme.xaml` — color brushes, gradient backgrounds, nav-button styles, card style.

Why DPAPI for token storage

Windows Data Protection API (DPAPI) encrypts data with a key derived from the user's credentials. Tokens encrypted with DPAPI cannot be decrypted by another user on the same machine and cannot be decrypted at all outside the local machine. This is a better fit than the Windows credential manager for ephemeral secrets; credentials go to the credential manager, access tokens to DPAPI.

Conversational view construction

The MessagesPage, added in 1.7.5, is a good example of WPF data binding plus code-behind for dynamic content. The XAML declares a two-column `Grid` with a `ListBox` on the left (threads) and a conversation pane on the right. Because each message bubble must be coloured differently depending on sender, the bubbles are built in code-behind (`BuildBubble`) rather than via a XAML `DataTemplate` with converters — this avoids having to ship four value converters for a decorative layout.

Building the Windows client

Open `windows/MedPharm/MedPharm.sln` in Visual Studio 2022 or later and build, or use `dotnet build` on the command line on Windows. Cross-building from Linux uses `true` in the project file, but the resulting binaries will still need to be signed and tested on Windows.

PART

V

Cross-Cutting Concerns

Audit, encryption at rest, TLS, Docker, Kubernetes — the systems that span every layer and determine how the whole product behaves in production.

CHAPTER 22

Audit Logging and PHI Access Trails

Audit is not a feature bolted on — it is an invariant maintained by every code path that changes state or views PHI. This chapter describes the two logging streams, the helper methods that emit to them, and the rules you must follow when adding new routes.

Two streams

MedPharm emits audit events to two separate tables: `audit_logs` for state changes, and `phi_access_logs` for read-only PHI access. The separation matters because retention, review, and export rules differ. An auditor asking "who modified anything last Thursday?" wants audit_logs; one asking "who looked at this patient's chart?" wants phi_access_logs.

The log_action helper

```
def log_action(self, user_id: int, action: str, entity_type: str = "",
 entity_id: int = None, details: dict = None, ip: str = ""):
 with self.get_session() as session:
 row = AuditLog(
 user_id=user_id,
 timestamp=datetime.utcnow(),
 action=action,
 entity_type=entity_type,
 entity_id=entity_id,
 details_json=json.dumps(details or {}, default=str),
 ip_address=ip,
 )
 session.add(row)
```

Listing 22-1. Canonical audit emission.

Idioms for action names

Action names follow a verb-noun convention in snake_case: `create_patient`, `update_prescription`, `delete_note`, `submit_claim`, `process_payment`. PHI-read actions use the `view_` prefix: `view_patient_chart`, `view_prescription`. Consistency matters — auditors grep for these strings.

What to include in details_json

- For create: the new record's id and a small summary (patient name, rx number, etc.). Do not include the full object.
- For update: only the changed fields, in `{"before": {...}, "after": {...}}` form.

- For delete: the id of the deleted row and a reason if the caller supplied one.
- Never include raw passwords, token strings, Fernet keys, or MFA secrets. These are all audit-log poison.

Caution. A common mistake is to log the full form data posted by the user. Form data can contain fields that were edited and immediately corrected — the audit trail should reflect the change the database actually stored, not the transient values typed along the way.

PHI access logging

Routes that read PHI should call `self.log_phi_access(...)` on the DatabaseManager with the viewing user, the patient_id, the record type, and a short reason. The pattern is not yet uniformly applied across every endpoint — a pull request that adds missing calls is always welcome — but new endpoints are expected to honour it.

Defence-in-depth helpers (1.7.6)

MedPharm 1.7.6 adds three production-grade defences. They are first-class members of the security package and are imported the same way as `@login_required`:

```
from security import (
 safe_harbor, redact_text, # security.deidentify
 RateLimiter, flask_rate_limit, # security.rate_limit
 install_phi_redaction_filter, # security.log_redaction
)

# Safe Harbor de-identification (45 CFR § 164.514(b)(2))
de_identified = safe_harbor(patient_record)
clean_text = redact_text(clinical_note)

# Per-IP rate limit on the unauthenticated portal endpoints
rl = RateLimiter()
@portal_bp.route("/login", methods=["POST"])
@flask_rate_limit(rl, scope="login", capacity=10, window_seconds=60)
def login(): ...

# Defence-in-depth PHI scrub on application logs
import logging
install_phi_redaction_filter(logging.getLogger())
```

Listing 22-2. Importing and applying the defence-in-depth helpers.

Hardened defaults

- `security/encryption.py` refuses production boot when the cryptography package is not installed or `MEDPHARM_FIELD_KEY` is unset. The fallback HMAC+XOR construction is for development only.

- `security/sessions.py` adds an absolute session-lifetime cap (default 8 × idle = 2 h) separate from the idle timeout. Tunable via `MEDPHARM_ABSOLUTE_SESSION_LIFETIME`.
- Optional strict CSP (drop 'unsafe-inline' for scripts) via `MEDPHARM_STRICT_CSP=1`. Opt-in because the bundled templates contain inline <script> tags; switch on after auditing them.
- `security/config.py` surfaces the new knobs as `absolute_session_lifetime_seconds`, `strict_csp`, and `rate_limit_per_minute`.

CHAPTER 23

TLS and Certificate Lifecycle

Every installation path terminates TLS by default: Docker, source Python, Kubernetes, and the full-stack image all ship TLS-capable. The Service Manual documents the operator-facing workflow in detail; this chapter covers the developer's view — where the code that runs the TLS logic lives, and how to make changes safely.

The three modes

`MEDPHARM_TLS_MODE` takes three values:

- **auto** — default. If a cert is present in `MEDPHARM_TLS_DIR`, use it; otherwise generate a self-signed one on first boot.
- **require** — fail to start unless a cert is present. Use this in production to guarantee no plaintext fallback.
- **disable** — serve plaintext. Local development only; never in production.

Generation script

`server/nginx/generate-cert.sh` is the single source of truth for self-signed cert generation. It produces an RSA-4096 cert valid for `MEDPHARM_TLS_DAYS` days (default 825) with the common name set from `MEDPHARM_TLS_HOSTNAME`. The same script is invoked from the Docker entrypoint, from `./start_cloud.sh`, and from the Kubernetes wrapper.

Bringing your own cert

To use a CA-issued cert, drop `fullchain.pem` and `privkey.pem` into the `medpharm-tls` Docker volume or the `medpharm-tls` Kubernetes Secret, set `MEDPHARM_TLS_MODE=require`, and restart. The gunicorn and Nginx configs both honour the two filenames.

Developer convenience

For local dev, the easiest path is `MEDPHARM_TLS_MODE=disable ./start_cloud.sh`, which serves plaintext HTTP on port 8080. Mobile clients in the emulator/simulator work either way — the Android network security config trusts user-installed CAs on localhost, and the iOS simulator accepts a self-signed cert once added to the trust store.

CHAPTER 24

Docker and Containerisation

Two images are published to Docker Hub under the enlightec namespace:

- `enlightec/medpharm-api` — API-only. Built from the root `Dockerfile`. Exposes port 8080.
- `enlightec/medpharm-server` — full stack (Nginx + API + web portal under supervisord). Built from `server/Dockerfile`. Exposes 80, 443, 8080, 5000.

Dockerfile structure

Both Dockerfiles are multi-stage, pinned to Ubuntu 24.04 LTS, and follow the same shape:

- Install system dependencies (Python, Nginx, build tools required for `cryptography`).
- Create an unprivileged `medpharm` user.
- Copy the source tree.
- Install Python requirements into a virtualenv under `/opt/medpharm/venv`.
- Set sane ENV defaults (ports, TLS mode, worker counts); the database is selected at run time by MEDPHARM_DATABASE_URL and defaults to a SQLite file under /data when it is unset.
- Declare exposed ports, volumes, health check, and entrypoint.

Supervisord for the full-stack image

`server/supervisord.conf` manages three processes: Nginx, gunicorn-for-API, and gunicorn-for-web-portal. Each has its own log destination. When the container exits, supervisord stops all three gracefully. Log rotation is delegated to the host or orchestrator; within the container, logs stream to mounted volumes.

Compose and the shared database

`docker-compose.yml` brings up two services: a `db` service running `postgres:16-alpine` with a `medpharm-pgdata` volume and a `pg_isready` health check, and the `api` service, which declares `depends_on: db (condition: service_healthy)` and reaches it through `MEDPHARM_DATABASE_URL=postgresql+psycopg://medpharm:${MEDPHARM_DB_PASSWORD}@db:5432/medpharm`. This is the shared-backend default: every client that points at this URL reads and writes one database. Commenting the URL out (and uncommenting `MEDPHARM_DB_PATH`) falls the API back to a private SQLite file under the `medpharm-data` volume. Supply `MEDPHARM_DB_PASSWORD` in the environment or an `.env` file rather than accepting the `change-this-in-production` placeholder.

Building locally

```
# API only
docker build -f Dockerfile \
 -t enlightec/medpharm-api:1.7.6 \
 -t enlightec/medpharm-api:latest .

# Full stack
docker build -f server/Dockerfile \
 -t enlightec/medpharm-server:1.7.6 \
 -t enlightec/medpharm-server:latest .

# Test locally
docker compose -f docker-compose.hub.yml up -d
curl -k https://localhost:8080/api/v1/health
```

Listing 24-1. Local build and quick-launch.

CI/CD

`.github/workflows/docker-publish.yml` builds and pushes both images on every `v*.*.*` tag push. The workflow validates Python syntax, exercises the database initialisation code path, verifies the API health endpoint, and only then runs the Docker build. Credentials come from repository secrets `DOCKERHUB_USERNAME` and `DOCKERHUB_TOKEN`.

> **Tip.** If CI is blocked (billing, runner outage) the manual fallback is to build and push from a developer machine — it is the same two `docker build` commands followed by four `docker push`es. Only run this with explicit release-manager authorisation; tagged images on Docker Hub are a contract.

Convergence with the native systemd install

Both Dockerfiles deliberately mirror what `install-services.sh` sets up on a bare-metal host: a system user named `medpharm` (UID 1000, no login shell, home `/opt/medpharm`), a Python venv at `/opt/medpharm/venv`, source files `COPY`ed under `/opt/medpharm/`, and the same set of `MEDPHARM_*` environment variables. The user's shell is `/usr/sbin/nologin` in both flavours so an exploit that opens an interactive shell as the medpharm user cannot get a real login. The Kubernetes pod (Chapter 25) carries this through with `runAsUser: 1000` and `runAsGroup: 1000`.

When you change one of these conventions you must change all three places — the Dockerfiles, the systemd unit templates at `systemd/medpharm-{api,web}.service.template`, and the k8s deployment at `k8s/base/deployment.yaml` — or the security posture across the deployment matrix will diverge in a way that is easy to miss in a pull request review.

CHAPTER 25

Kubernetes Deployment

Kubernetes manifests live under `k8s/` as a Kustomize base with three overlays: `overlays/gcp/`, `overlays/aws/`, and `overlays/generic/`. The wrapper script `k8s/medpharm-k8s.sh` abstracts the common workflows — install, status, deploy, backup, rotate-tls.

Base manifests

- `postgres-statefulset.yaml` — the `medpharm-postgres` StatefulSet (postgres:16-alpine, uid/gid 70, a 5 Gi `volumeClaimTemplate`, and `pg_isready` probes). The shared database every component attaches to.
- `postgres-service.yaml` — the headless `medpharm-postgres` Service on 5432 that the app and the init-container resolve.
- `deployment.yaml` — deployment of the `enlightec/medpharm-server` image. Because every replica shares the one networked database it is no longer single-writer-bound: the strategy is `RollingUpdate` and `replicas` may be raised past 1. An `initContainer` blocks on `pg_isready` so the app does not crash-loop racing Postgres on a cold start.
- `service.yaml` — ClusterIP service exposing 80 and 443.
- `ingress.yaml` — ingress rules; overlays supply the cloud-specific ingress class (GCE, ALB, nginx).
- `secret.example.yaml` — template Secret `medpharm-secrets` carrying MEDPHARM_JWT_SECRET, MEDPHARM_SECRET_KEY, MEDPHARM_DB_PASSWORD, and the credential-bearing MEDPHARM_DATABASE_URL. Operators fill in real values via `kubectl create secret generic`; the URL lives only in the Secret, never the ConfigMap.
- `configmap.yaml` — non-secret env `medpharm-config` (ports, workers, CORS, and the SQLite `MEDPHARM_DB_PATH` fallback).
- `pvc.yaml` — PersistentVolumeClaim for `/data` and `/etc/ssl/medpharm`.
- `kustomization.yaml` — lists the above and sets the container image tag.

Scaling story

The Kubernetes base now ships PostgreSQL as the shared backend, so the historical single-writer ceiling is gone. The app Deployment defaults to one replica purely for economy; because every pod reads and writes the one networked database through `MEDPHARM_DATABASE_URL`, write throughput scales by raising `replicas` — or by attaching a HorizontalPodAutoscaler to the Deployment — with no code change. The earlier advice to never run multiple SQLite pods still holds for the SQLite default; it simply no longer applies once the Postgres StatefulSet is the backend. The database itself

remains a single StatefulSet replica, which is the next axis to scale (read replicas, a managed Postgres service) should it become the bottleneck.

Hot backup

`k8s/medpharm-k8s.sh backup` runs SQLite's online backup API against the running pod and streams the result to the local file. The database remains available throughout; there is no lock held longer than a few milliseconds.

Pod and container security context

The deployment hardens the pod with `runAsNonRoot: true`, `runAsUser: 1000`, `runAsGroup: 1000`, `fsGroup: 1000` (matching the medpharm user baked into the image), and the kubelet's `seccompProfile: RuntimeDefault`. The container drops every Linux capability with `capabilities.drop: [ALL]` and only adds `NET_BIND_SERVICE` back so nginx can bind :80/:443 — operators who front the deployment with an external ingress (and therefore only need :8080/:5000 out of the pod) can drop NET_BIND_SERVICE entirely. `allowPrivilegeEscalation: false` ensures setuid binaries cannot regain privileges, and the read-only rootfs flag is left at `false` only because Nginx writes its PID file under `/var/run`; moving Nginx to write under a tmpfs would let you flip this on too.

PART

VI

Development Workflow

How to get a working build, what tests to run, how to land a change, how releases are cut, and what to do when something breaks.

CHAPTER 26

Setting Up a Development Environment

A fresh checkout to a working clinical desktop should take under ten minutes on Ubuntu or macOS and under fifteen on Windows (via WSL2). This chapter walks through the exact sequence.

Prerequisites

Tool	Minimum version	Needed for
Python	3.10	Everything server-side
Git	2.30	Source control
Docker	24+	Container builds (optional)
Xcode	15	iOS/macOS builds
Android Studio	Iguana (2023.3)	Android builds
.NET SDK	8.0	Windows builds
Qt6	(via pip)	Desktop UI

The canonical install

```
git clone https://github.com/stillwell/MedPharm.git
cd MedPharm
chmod +x install.sh
./install.sh
```

Listing 26-1. The happy path.

The install script detects the OS, installs system dependencies, creates a virtualenv under `venv/`, installs Python requirements, initialises the database (a local SQLite file unless `MEDPHARM_DATABASE_URL` points at a shared PostgreSQL server), and runs the seed scripts. On completion, three launchers are ready: `./start_cloud.sh`, `./start_web.sh`, and `./start_desktop.sh`.

Manual install (if the script fails)

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
pip install -r requirements-cloud.txt
pip install reportlab pygame # optional extras
```

```
python3 -c "
from database.db_manager import DatabaseManager
from database.seed_data import seed_database
from database.seed_expanded import seed_expanded_data
db = DatabaseManager('medpharm_erp.db')
db.init_db()
seed_database(db)
seed_expanded_data(db)
print('DB ready')
"
```

Listing 26-2. Manual install path.

Environment variables worth setting

Variable	Dev default	Purpose
MEDPHARM_TLS_MODE	disable	Plaintext HTTP during local dev
MEDPHARM_DEBUG	true	Flask debug mode; autoreload and verbose errors
MEDPHARM_JWT_SECRET	(dev-default)	Override to test refresh logic explicitly
MEDPHARM_FIELD_KEY	(auto-generated)	For testing encryption rotation
MEDPHARM_DB_PATH	medpharm_erp.db	SQLite file used when MEDPHARM_DATABASE_URL is unset; point at a
MEDPHARM_DATABASE_URL	(unset)	Full SQLAlchemy URL of a shared backend, e.g. postgresql+psycopg://m
MEDPHARM_DB_PASSWORD	(unset)	Postgres password the Docker/k8s URLs interpolate; set alongside a cont

Editor configuration

The project does not ship an editor configuration, but all Python files are PEP 8 formatted with two small tolerances: lines may exceed 79 columns up to 100, and double-quoted strings are the norm. The Swift code is Swift-Format-defaulted. Kotlin uses the Android Studio defaults. C# uses the Visual Studio / Rider defaults. Reformat anything you touch if it deviates.

Building the iOS and macOS clients from a non-Mac host

Apple's toolchain is macOS-only — `xcodebuild` does not exist on Linux or Windows, the SDK is not redistributable, and Apple's code-signing identities and notarisation service are macOS-only. Engineers without a Mac on the bench therefore use one of three paths, each wired into helper scripts that live next to the Apple source.

Path	What it produces	When to use
GitHub Actions	MedPharm-iOS-Simulator.app.zip + an unsigned device archive, or MedPharm-macOS.app.zip	Default. Build runs on a macOS-14 runner; signed .ipa requ
Cirrus CI fallback	Same artifacts as GitHub Actions	When the GitHub account is billing-blocked from macOS n
Swift on Linux compile-check	Library build of the Foundation-only subset (Models + QRConfigParser + DrugInfoURL)	Fast feedback when editing; cannot compile Views/ (S

The CI workflows live at `.github/workflows/ios-build.yml` and `.github/workflows/macos-build.yml`. The Cirrus config is `.cirrus.yml` at the repo root and requires a one-time install of the Cirrus CI GitHub App. Both CI paths skip the build when no source under `ios/` or `macos/` has changed.

Linking out to authoritative drug information

All five clients (iOS, macOS, Android, Qt desktop, web portal) include a small DrugInfoURL helper that builds a MedlinePlus search URL from a Medication's brand or generic name and returns `nil` for blank input. Tapping a medication row (or the brand-name header on the web) opens the resulting URL in the user's default browser. The MedlinePlus search URL must hit the NLM vsearch backend at `vsearch.nlm.nih.gov/vivisimo/cgi-bin/query-meta` with the `v:project=medlineplus` and `v:sources=medlineplus-bundle` parameters; the shorter `medlineplus.gov/search.html` URL returns a 404. The helper also exposes DailyMed (FDA labels) and Drugs.com as alternatives for clinicians who want either the prescribing information or a layperson summary.

ngrok-tunnel safety net for non-Mac access

When a deployment's API is fronted by an ngrok tunnel (see `./start_ngrok.sh` and the docker-compose `ngrok` profile), every mobile and desktop client adds the `ngrok-skip-browser-warning: true` header to its outgoing requests. This bypasses the free-tier ngrok HTML interstitial that would otherwise land an HTML page in the JSON parser on the very first request from a fresh install. The header is harmless when ngrok is not in front of the API, so it ships unconditionally.

Production deployment from a development checkout

Development happens in the engineer's home-directory checkout; production deployment lives at `/opt/medpharm` under the `medpharm` system user. The script that bridges the two is `install-services.sh`. It performs an idempotent six-step migration of the current checkout (or, with `--clone-fresh`, a fresh clone from origin) into `/opt/medpharm`, builds a venv in place, and renders the two systemd unit templates from `systemd/` into `/etc/systemd/system/`.

Stage	What happens
Pre-flight	Templates present, systemctl on PATH, python3 available

Stage	What happens
User	useradd -r -s /usr/sbin/nologin -d /opt/medpharm -M -U medpharm; skipped if it already exists
rsync	$SCRIPT_DIR → /opt/medpharm with .git preserved (so update.sh can fast-forward in place) and venv/, __py
venv	/opt/medpharm/venv via python3 -m venv; pip install -r requirements-cloud.txt (and best-effort requirements.t
Permissions	chown -R medpharm:medpharm; chmod 750 prefix, 770 data + logs
Units	Template substitution → /etc/systemd/system/, daemon-reload, enable, start (skip with --no-start)

The unit templates enable the standard hardening posture — `NoNewPrivileges`, `ProtectSystem=strict`, `ProtectHome=read-only` with explicit `ReadWritePaths`, `MemoryDenyWriteExecute`, `RestrictNamespaces`, `SystemCallFilter=@system-service` — and route stdout / stderr to journald, queryable via `journalctl -u medpharm-api -u medpharm-web -f`. Operator overrides go in `/etc/medpharm/medpharm.env` (system-wide) or `${INSTALL_DIR}/.env` (per-install); both are loaded with the `EnvironmentFile=-` syntax so their absence is not an error.

How update.sh keeps a deployed checkout current

Once a host is running off `/opt/medpharm`, `./update.sh` inside that tree handles fast-forward updates from origin. The interactive flow has been there for some time; what is new in 1.7.6-C is the unattended path. `./update.sh --auto` is the cron / systemd-timer entry point: it acquires a directory lock at `.update.lock.d/` (with PID staleness recovery) so it cannot collide with a manual run, refuses to touch a dirty working tree (auto-stashing without supervision is a foot-gun), runs the SQLite online `.backup` against the autodetected database path before any code changes, and prints a single summary line to stdout only when an update was actually applied — so a weekly timer that runs ten times for ten unchanged weeks produces no email noise.

`--install-schedule[=PERIOD]` installs that auto-run as a recurring job. The installer prefers a systemd `--user` timer — sandboxed to the project tree, persistent across reboots with `Persistent=true` (catches up after wake), with a `RandomizedDelaySec=10min` jitter — and falls back to a crontab entry on hosts without a user manager. Both flavours carry a `# medpharm-auto-update` marker so `--uninstall-schedule` removes precisely the entry the installer created. PERIOD accepts the friendly names `hourly|daily|weekly|monthly` plus the passthrough forms `OnCalendar=…` (systemd) and `cron=…` (cron) for advanced cadences.

Real bug fix worth flagging. Before 1.7.6-C the `backup_database` function in update.sh hard-coded `DB_PATH=data/medpharm.db`, a file that does not exist in any of the supported deployment topologies. The consequence: every "Backing up database…" line in the log was followed by a silent skip. The autodetect now probes, in order, `$MEDPHARM_DB_PATH` → `medpharm_erp.db` → `data/medpharm_erp.db` → `data/medpharm.db`, which catches the run_cloud.py default, the docker-compose volume mount default, and the legacy path. This online `.backup` is the SQLite path only; a deployment that has moved its data to the shared PostgreSQL backend

(`MEDPHARM_DATABASE_URL`) backs up at the database server instead — with `pg_dump` or a managed-service snapshot — and the autodetect simply finds no file to copy. Operators who relied on the previous behaviour for any reason should consult the script before upgrading.

CHAPTER 27

Running Tests and Smoke Checks

MedPharm does not currently maintain an exhaustive unit test suite — an honest statement that should be understood as "there is room to help." What does exist is a deliberate set of smoke checks that exercise the application's critical paths end-to-end. Passing them is the bar a pull request must clear.

The CI smoke suite

The GitHub Actions workflow `.github/workflows/docker-publish.yml` runs the following in sequence on every tag push:

- `python -m py_compile` against every module under `api/`, `database/`, and `web/`. Catches syntax errors.
- Database initialisation — creates a throwaway SQLite file, runs both seed scripts. Catches ORM misconfiguration.
- API app creation — constructs the Flask application and queries `/api/v1/health`. Catches blueprint registration bugs.
- Web portal app creation — renders `/login`. Catches template errors.
- Docker image build — end-to-end container bring-up. Catches Dockerfile mistakes.

Running smoke checks locally

```
source venv/bin/activate
python3 -m py_compile api/app.py api/auth.py api/routes.py \
 database/models.py database/db_manager.py \
 web/app.py web/routes.py

python3 -c "
import sys, os; sys.path.insert(0, os.getcwd())
from database.db_manager import DatabaseManager
from database.seed_data import seed_database
from api.app import create_cloud_app
import tempfile
tmp = tempfile.mktemp(suffix='.db')
db = DatabaseManager(tmp); db.init_db(); seed_database(db)
app = create_cloud_app(db); client = app.test_client()
r = client.get('/api/v1/health'); assert r.status_code == 200
os.unlink(tmp); print('API OK')
"
```

Listing 27-1. The minimum pre-commit smoke sequence.

End-to-end tests for new features

When adding a feature that crosses the wire, write a short end-to-end test script that logs in, exercises the happy path, and verifies the expected DB state. The messaging feature (1.7.5) was validated this way before merge — see the commit message for that release for a concrete example.

Client-side testing

Each native client has standard platform tooling: XCTest on iOS/macOS, JUnit/Espresso on Android, xUnit on .NET. None is currently integrated into the CI pipeline; contributing a scaffold for any of them is an open invitation. In the meantime, clients are validated by manual testing against a running backend.

CHAPTER 28

Branching, Commits, and Pull Requests

The project uses a plain single-main-branch workflow. Feature branches merge to `master` via pull request; releases are cut by tagging `master` with `v1.7.x`.

Commit message conventions

- Subject line ≤ 70 characters, imperative mood.
- Prefix with a subsystem or area name when helpful: `Qt:`, `api:`, `android:`, etc.
- Body paragraphs explain *why*, not *what* — the diff shows what.
- End with a `Co-Authored-By:` trailer for pair-programmed or AI-assisted work.

Pull request checklist

- Smoke checks (Chapter 27) pass locally.
- New or changed routes are documented in Appendix C (endpoint catalogue).
- New models include a migration SQL file in `docs/migrations/` if they add columns to existing tables.
- PHI-touching code paths emit audit and PHI access logs.
- Secrets are not committed. `.env` and `*.key` files are .gitignored; verify before pushing.
- The PR description includes a test plan.

CHAPTER 29

Release Engineering

Releases bump the version, rebuild the PDFs, push to Docker Hub, and push the tag to GitHub. The mechanics are mechanical; the judgement is in picking the right version increment.

Version policy

- Major (x.0.0) — incompatible API changes. Has not happened yet; when it does, `v2` blueprint coexists with `v1`.
- Minor (1.x.0) — new feature areas, additive API changes, new clients.
- Patch (1.7.x) — bug fixes and small additive changes that do not break older clients.

The release sequence

- Bump version strings — `__init__.py`, `run_qt.py`, `api/{app,routes,fhir}.py`, `install.sh` banner, `Dockerfiles`, `SECURITY.md`, HIPAA doc header, `android/app/build.gradle.kts`, iOS/macOS version strings, Windows `.csproj`, k8s defaults, the three PDF generators.
- Do not bump — kotlinx-coroutines library version (it happens to equal 1.7.x), historical references in the service manual.
- Add the new tag to the supported-tags list in `README.md` without removing older tags.
- Regenerate all three PDFs.
- Commit everything with a `Release X.Y.Z:` subject.
- Tag with `git tag -a vX.Y.Z -m '...'`.
- Push `master` and the tag.
- Verify CI built the Docker images. If CI is blocked, build and push manually (see the Docker chapter).

CHAPTER 30

Debugging Techniques and Tooling

Debugging is the skill the manual cannot teach but can scaffold. This chapter lists the techniques that have proven most useful on MedPharm, arranged by the subsystem they apply to.

The SQL dump

When an ORM query produces surprising results, the first thing to do is see the SQL. Set `echo=True` on the engine temporarily in `DatabaseManager.init_db` and re-run. SQLAlchemy will print every statement, and the cause of an N+1 or a missing filter is usually obvious after fifteen seconds of log scanning.

The HTTP replay

When a client bug appears to be server-side, capture the actual request with `curl -v` against the running backend — the Android app's `OkHttp` logging interceptor and the iOS `URLSession` console output both give the URL, headers, and body. A reproducible `curl` makes subsequent debugging mechanical.

The debugger

The Python debugger (`import pdb; pdb.set_trace()` or, on 3.7+, `breakpoint()`) works fine in development. In Qt, it does not play well with the event loop; for Qt debugging, prefer structured logging or a remote debugger from an IDE. For mobile clients, use the platform debugger — Xcode's or Android Studio's.

Responsible disclosure of security issues

If you discover a security vulnerability, do not open a public GitHub issue. Write to Andrew.Stillwell@enlightec.com with subject prefix `[security]` and a private description of the issue. Expect an acknowledgement within one business day; a fix timeline will be coordinated privately. Disclosure to third parties is appropriate only after a fix has shipped and affected operators have been notified.

PART

VII

Extending MedPharm

Recipes for the four most common expansion tasks: a new model, a new endpoint, a new clinical module, and a new client platform.

CHAPTER 31

Recipe — Adding a New Model

When to reach for this recipe: a new table is needed to store a concept that does not fit an existing table. Not when an existing table needs a new column — that is a column-add and a migration, not a model addition.

Step 1 — Design the columns

Before writing any code, list the columns. For each, ask: is this PHI? Is it queryable? Is it a finite set (→ enum)? Does it reference another table (→ ForeignKey + relationship)? The Service Manual's Part II has a useful checklist.

Step 2 — Add the model class

Place the class in the topically appropriate group in `database/models.py`. If you are adding a new group entirely, add a banner comment. Remember to add a `relationship()` on both sides of any `ForeignKey`.

Step 3 — Extend the import list in db_manager

The top of `database/db_manager.py` imports every model explicitly. Add your new class there so that queries can refer to it.

Step 4 — Add DatabaseManager methods

Typical new methods: `create_`, `get_`, `list_s`, `update_`, `delete_`. Each opens a `get_session()` contextmanager and returns plain-dict data where possible.

Step 5 — Migration SQL (if live deployments exist)

Produce `docs/migrations/YYYYMMDD_add_.sql` with the `CREATE TABLE` and any supporting indices. Operators apply migrations manually on existing deployments.

Step 6 — Wire it to the API

See Chapter 32.

Step 7 — Wire it to the clients

If the new thing is patient-visible, add it to the web portal and the four native clients. Chapter 34 covers the client-side recipe.

CHAPTER 32

Recipe — Adding a New API Endpoint

You have a DatabaseManager method and want to expose it over HTTP. This is the shortest recipe in the manual.

Step 1 — Pick a URL

- Under `/patient/` if a patient is the caller.
- Under `/staff/` if a staff member is the caller.
- Plural collection, singular detail. Verbs as actions at the leaf.

Step 2 — Write the route

```
@api_bp.route("/staff/widgets", methods=["GET"])
@staff_required
def staff_list_widgets():
 widgets = g.db_manager.list_widgets()
 return jsonify({"widgets": widgets})


@api_bp.route("/staff/widgets", methods=["POST"])
@staff_required
def staff_create_widget():
 data = request.get_json(silent=True) or {}
 name = (data.get("name") or "").strip()
 if not name:
 return jsonify({"error": "Name required"}), 400
 widget_id = g.db_manager.create_widget(
 name=name, created_by=g.current_user_id)
 return jsonify({"widget_id": widget_id}), 201
```

Listing 32-1. A complete pair of CRUD endpoints.

Step 3 — Document in Appendix C

Every new endpoint must land in the API endpoint catalogue (Appendix C) in the same pull request. An endpoint that is not documented is an endpoint the clients cannot find.

Step 4 — Smoke-test

Add a one-liner to the smoke check (Chapter 27) that exercises the new endpoint against a throwaway database. Takes a minute; prevents the new endpoint from silently breaking in a future refactor.

CHAPTER 33

Recipe — Adding a New Clinical Module

A clinical module is a larger piece of functionality — a new navigation destination in the Qt app, a corresponding page in the web portal, API endpoints, and client support on the native platforms. The messaging feature (1.7.5) is the canonical recent example and a good template.

Step 1 — Model and DB methods

Follow Chapter 31 to add whatever tables and DatabaseManager methods the module needs.

Step 2 — API routes

Follow Chapter 32 for each endpoint. Group related routes under a banner comment in `api/routes.py`.

Step 3 — Qt widget

- Create `qt_app/widgets/_widget.py`.
- Import it in `qt_app/main_window.py`.
- Add a tuple to `NAV_ITEMS`.
- Add a `self.stack.addWidget(...)` line in the same order.

Step 4 — Web portal page

- Add a route function to `web/routes.py`.
- Add a template under `web/templates/`. Extend `base.html`.
- Add a nav link to `base.html`.

Step 5 — Native clients

See Chapter 34 for the client-side recipe. Add the feature to Android, iOS, macOS, and Windows in the same pull request or a follow-up labelled with the same feature name.

Tip. If the feature is large, land it in phases: backend + Qt first (one PR), web portal next, each mobile/desktop client in its own PR. This reduces review surface and reduces rollback risk.

CHAPTER 34

Recipe — Adding a New Client Platform

Adding a whole new client platform — say, a tvOS patient check-in kiosk, or a Rust command-line client for operators — is a larger undertaking than adding a feature to an existing client. This recipe sketches the work.

Step 1 — Minimum viable client

- HTTP client library with TLS.
- Secure storage for the access and refresh tokens (platform-appropriate: Keychain, KeyStore, DPAPI, etc.).
- Model types matching the API JSON shapes.
- At minimum: login, one happy-path screen, logout.

Step 2 — Server-URL override

Every existing client exposes a `Server URL` field on the login screen with a "Reset to default" button. Honour this convention — operators rely on it for pointing demo builds at staging backends.

Step 3 — Network security

- Require TLS by default.
- Provide a documented path for accepting a self-signed cert during development (e.g. user-installed CA on Android, trust store modification on iOS).
- Never ship with certificate verification disabled; when tempted, add a developer-only flag behind a build configuration rather than hardcoding.

Step 4 — Build and packaging

Document the build steps in `docs/COMPILATION.md` alongside the existing platforms. Include platform-specific signing requirements.

Step 5 — Feature parity

Once the client is booted, working backwards through the feature list from the nearest existing client — Android if the new platform is mobile, Windows if it is desktop — is the fastest way to approach parity. The existing clients already provide the reference UX flows; your job is to translate them into idiomatic code for the new platform.

Step 6 — Update the manual

Add a chapter to Part IV of this manual describing the new client's layout, idioms, and quirks. A new client without a corresponding manual chapter is an undocumented liability.

PART

VIII

Appendices

The reference tables. File tours, schema listings, endpoint catalogues, error codes, glossary, and contact information.

APPENDIX A

File-by-File Code Tour

This appendix is the reference you reach for when someone asks "where does the code for X live?". Files are grouped by area; for each, the one-line purpose and the most important classes or functions are listed.

Database layer

Path	Purpose / key symbols
database/models.py	All 50+ SQLAlchemy ORM classes and 20+ enums. Base class, relationships, table args.
database/db_manager.py	DatabaseManager. Session lifecycle, every CRUD, business logic that spans rows.
database/seed_data.py	Seeds baseline fixtures — users, patients, medications, interactions. Idempotent.
database/seed_expanded.py	Seeds reference data — symptoms (30+), conditions (25+), additional medications.

API layer

Path	Purpose / key symbols
api/app.py	create_cloud_app factory. CORS, security middleware, blueprint registration.
api/auth.py	JWT create/decode, @token_required, @patient_required, @staff_required, @admin_req
api/routes.py	api_bp blueprint. All /api/v1/* endpoints.
api/fhir.py	FHIR R4 CapabilityStatement and resource adapters.

Security layer

Path	Purpose / key symbols
security/encryption.py	FieldCipher. encrypt_field / decrypt_field. Key rotation.
security/audit.py	Helpers to emit AuditLog and PHIAccessLog rows from middleware.
security/csrf.py	csrf_required decorator for web portal.
security/lockout.py	LockoutTracker. AccountLockedError.
security/passwords.py	PasswordPolicy. validate_password. Password reuse check.

Path	Purpose / key symbols
security/sessions.py	Flask-session lifecycle; session regeneration on auth.
security/totp.py	TOTP generate/verify; enrollment URL; backup codes.
security/emergency.py	Break-glass grant, verify, expire, review.
security/phi.py	Catalogue of PHI columns. Used by audit helpers.
security/config.py	SecurityConfig dataclass; env-driven tunables.

Qt desktop

Path	Purpose
qt_app/main_window.py	MainWindow, NAV_ITEMS, navigation stack.
qt_app/styles.py	Global QSS stylesheet.
qt_app/dialogs/login_dialog.py	Staff login modal.
qt_app/widgets/dashboard_widget.py	KPIs, today's activity.
qt_app/widgets/patient_widget.py	Master-detail patient panel + Notes tab.
qt_app/widgets/prescription_widget.py	Prescription list + creation dialog with DDI check.
qt_app/widgets/medication_widget.py	Formulary browser.
qt_app/widgets/appointment_widget.py	Calendar scheduling.
qt_app/widgets/records_widget.py	Medical records list + new-record dialog.
qt_app/widgets/billing_widget.py	Invoices, payments, insurance claims, processing.
qt_app/widgets/messages_widget.py	Secure messaging (staff side).
qt_app/widgets/symptoms_widget.py	Symptoms + conditions reference browser.
qt_app/widgets/analytics_widget.py	matplotlib-powered charts.

Web portal

Path	Purpose
web/app.py	create_app factory.

Path	Purpose
web/routes.py	portal_bp. Login, register, dashboard, prescriptions, billing, records, appointments,
web/templates/*.html	Jinja2 templates; extend base.html.
web/static/css/style.css	Dark-gradient theme.
web/static/vendor/*	Bootstrap, Font Awesome, Google Fonts.

Native clients

Path root	Language / purpose
android/	Kotlin; Jetpack / Retrofit MVVM.
ios/MedPharm/	Swift; SwiftUI / async-await.
macos/MedPharm/	Swift; SwiftUI / NavigationSplitView.
windows/MedPharm/	.NET 8; WPF MVVM.

Ops / deployment

Path	Purpose
Dockerfile	API-only image (Ubuntu 24.04 LTS).
server/Dockerfile	Full stack (Nginx + API + portal + supervisord).
server/supervisord.conf	Process manager for the full-stack image.
server/nginx/	Nginx configs and TLS generation.
docker-compose.yml	Local build-from-source compose.
docker-compose.hub.yml	Pull-from-Docker-Hub compose (API).
server/docker-compose.hub.yml	Pull-from-Docker-Hub compose (full stack).
install.sh	OS-detection installer.
uninstall.sh	Reverse installer.
start_*.sh	Quick-launch wrappers.
generate_docs.sh	Regenerates Volume I PDF.

Path	Purpose
k8s/	Kubernetes Kustomize base + cloud overlays.
.github/workflows/docker-publish.yml	CI/CD — build, test, push.

Documentation

Path	Purpose
docs/generate_pdf.py	Volume I (Technical Reference) generator.
docs/generate_service_manual.py	Volume II (Service Manual) generator.
docs/generate_developer_manual.py	Volume III (this manual) generator.
docs/API.md	REST API Markdown reference.
docs/CLIENTS.md	Multi-platform client notes.
docs/INSTALLATION.md	Install paths and troubleshooting.
docs/KUBERNETES.md	Kubernetes walkthrough.
docs/COMPILATION.md	Compilation instructions.
docs/SERVER.md	Full-stack server notes.
docs/HIPAA_COMPLIANCE.md	Compliance mapping.
docs/BAA_TEMPLATE.md	Business Associate Agreement template.
docs/BREACH_NOTIFICATION.md	Breach notification procedure.

APPENDIX B

Complete SQL Schema (Excerpted)

This appendix excerpts the `CREATE TABLE` statements that SQLAlchemy emits from `database/models.py`. Column types are shown in their SQLite equivalents; ORM-level types like `Enum` are rendered as `VARCHAR(32)` after SQLAlchemy maps them. Constraints and indices are preserved.

Core tables

```sql
CREATE TABLE users (
 id INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,
 username VARCHAR(50) NOT NULL,
 password_hash VARCHAR(256) NOT NULL,
 role VARCHAR(32) NOT NULL,
 first_name VARCHAR(100) NOT NULL,
 last_name VARCHAR(100) NOT NULL,
 email VARCHAR(200) NOT NULL,
 phone VARCHAR(20),
 license_number VARCHAR(50),
 specialization VARCHAR(200),
 created_at DATETIME,
 is_active BOOLEAN DEFAULT 1,
 UNIQUE (username),
 UNIQUE (email)
);
CREATE INDEX ix_users_username ON users (username);

CREATE TABLE patients (
 id INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,
 first_name VARCHAR(100) NOT NULL,
 last_name VARCHAR(100) NOT NULL,
 dob DATE NOT NULL,
 gender VARCHAR(32),
 ssn_last4 VARCHAR(4),
 email VARCHAR(200),
 phone VARCHAR(20),
 address VARCHAR(300),
 city VARCHAR(100),
 state VARCHAR(2),
 zip_code VARCHAR(10),
 emergency_contact_name VARCHAR(200),
 emergency_contact_phone VARCHAR(20),
 blood_type VARCHAR(32),
 created_at DATETIME,
 is_active BOOLEAN DEFAULT 1
```

```
);
CREATE INDEX ix_patients_name ON patients (last_name, first_name);
```

Listing B-1. users and patients tables.

Clinical tables

```
CREATE TABLE prescriptions (
 id INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,
 rx_number VARCHAR(30) NOT NULL UNIQUE,
 patient_id INTEGER NOT NULL REFERENCES patients(id),
 prescriber_id INTEGER NOT NULL REFERENCES users(id),
 status VARCHAR(32) DEFAULT 'pending',
 prescribed_date DATE,
 expiry_date DATE,
 notes TEXT
);

CREATE TABLE prescription_items (
 id INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,
 prescription_id INTEGER NOT NULL REFERENCES prescriptions(id),
 medication_id INTEGER NOT NULL REFERENCES medications(id),
 dosage VARCHAR(100) NOT NULL,
 frequency VARCHAR(100) NOT NULL,
 quantity INTEGER DEFAULT 0,
 refills_allowed INTEGER DEFAULT 0,
 refills_used INTEGER DEFAULT 0,
 instructions TEXT
);

CREATE TABLE appointments (
 id INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,
 patient_id INTEGER NOT NULL REFERENCES patients(id),
 provider_id INTEGER NOT NULL REFERENCES users(id),
 scheduled_datetime DATETIME NOT NULL,
 duration_minutes INTEGER DEFAULT 30,
 appointment_type VARCHAR(32),
 status VARCHAR(32) DEFAULT 'scheduled',
 reason TEXT,
 notes TEXT
);
CREATE INDEX ix_appointments_datetime ON appointments (scheduled_datetime);
```

Listing B-2. prescriptions, prescription_items, appointments.

Messaging and notes (1.7.5)

```
CREATE TABLE message_threads (
 id INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,
 patient_id INTEGER NOT NULL REFERENCES patients(id),
```

```
 provider_id INTEGER REFERENCES users(id),
 subject VARCHAR(200) NOT NULL,
 created_at DATETIME,
 last_message_at DATETIME,
 is_closed BOOLEAN DEFAULT 0
);
CREATE INDEX ix_message_threads_patient_id ON message_threads(patient_id);
CREATE INDEX ix_message_threads_provider_id ON message_threads(provider_id);
CREATE INDEX ix_message_threads_last_message_at ON
message_threads(last_message_at);

CREATE TABLE secure_messages (
 id INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,
 thread_id INTEGER NOT NULL REFERENCES message_threads(id),
 sender_type VARCHAR(20) NOT NULL,
 sender_id INTEGER NOT NULL,
 body_encrypted TEXT NOT NULL,
 sent_at DATETIME,
 read_at DATETIME
);

CREATE TABLE provider_notes (
 id INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,
 patient_id INTEGER NOT NULL REFERENCES patients(id),
 author_id INTEGER NOT NULL REFERENCES users(id),
 body_encrypted TEXT NOT NULL,
 is_pinned BOOLEAN DEFAULT 0,
 created_at DATETIME,
 updated_at DATETIME
);
CREATE INDEX ix_provider_notes_patient_id ON provider_notes(patient_id);
CREATE INDEX ix_provider_notes_author_id ON provider_notes(author_id);
CREATE INDEX ix_provider_notes_created_at ON provider_notes(created_at);
```

Listing B-3. Messaging and private-notes tables (added in 1.7.5).

APPENDIX C

API Endpoint Catalogue

Every /api/v1 endpoint is listed here with method, path, required role, and a one-line description. When you add or remove an endpoint, update this appendix in the same commit.

Authentication

Method	Path	Auth	Description
POST	/auth/login/patient	public	Patient login.
POST	/auth/login/staff	public	Staff login.
POST	/auth/register	public	Patient self-registration (4-factor).
POST	/auth/refresh	public	Exchange refresh for access token.

Health and reference

Method	Path	Auth	Description
GET	/health	public	Liveness probe.
GET	/medications/search	token	Search formulary.
GET	/medications/<id>	token	Medication detail.
GET	/reference/symptoms	token	Symptom search.
GET	/reference/symptoms/body-systems	token	List of body systems.
GET	/reference/conditions	token	Condition search.
GET	/reference/conditions/categories	token	List of condition categories.

Patient endpoints

Method	Path	Description
GET	/patient/dashboard	Dashboard KPIs and recent activity.
GET	/patient/profile	Patient demographics + insurance + allergies.
PUT	/patient/profile	Update phone/email/address.

Method	Path	Description
GET	/patient/prescriptions	List prescriptions (optional ?status=).
POST	/patient/prescriptions/<id>/refill	Request a refill.
GET	/patient/billing	Invoices, payments, balance.
POST	/patient/billing/<id>/pay	Submit payment.
GET	/patient/records	Medical records (optional ?type=).
GET	/patient/appointments	Upcoming and past appointments.
GET	/patient/medications	Current medication list.
GET	/patient/notifications	Aggregated notifications (upcoming, overdue).
GET	/patient/insurance	Patient's insurance records.
GET	/patient/insurance/claims	Claim history.
POST	/patient/insurance/claims	Submit a claim.
GET	/patient/messages	List message threads.
POST	/patient/messages	Start a new thread.
GET	/patient/messages/<id>	Read a thread (and mark read).
POST	/patient/messages/<id>/reply	Reply to a thread.
GET	/patient/messages/providers	List providers for compose dropdown.

Staff endpoints

Method	Path	Description
GET	/staff/dashboard	Staff dashboard stats and recent activity.
GET	/staff/patients	List/search patients.
GET	/staff/patients/<id>	Full patient record.
GET	/staff/appointments	Appointments by date or provider.
GET	/staff/prescriptions	Prescriptions (optional ?patient_id=).
GET	/staff/analytics/revenue	Revenue by month.

Method	Path	Description
GET	/staff/analytics/top-medications	Top prescribed medications.
GET	/staff/analytics/demographics	Patient demographics.
GET	/staff/insurance/claims	All claims (filterable).
POST	/staff/insurance/claims/<id>/process	Approve/deny/pay a claim.
GET	/staff/messages	All threads; filter by patient_id.
GET	/staff/messages/<id>	Read a thread (and mark read).
POST	/staff/messages/<id>/reply	Reply to a thread.
POST	/staff/messages/<id>/close	Close a thread.
GET	/staff/patients/<pid>/notes	List the current user's notes for this patient.
POST	/staff/patients/<pid>/notes	Create a private note.
GET	/staff/notes/<id>	Read a private note (if author).
PUT	/staff/notes/<id>	Update a private note (if author).
DELETE	/staff/notes/<id>	Delete a private note (if author).

APPENDIX D

HTTP Error Codes and Meanings

Clients should rely on the HTTP status code for programmatic behaviour and the JSON `error` message only for display. The table below summarises how each status is used across the API.

Status	Meaning in this API
200 OK	GET succeeded.
201 Created	POST that created a resource. Body includes the new id.
204 No Content	Rare; used for acknowledgements without a body.
400 Bad Request	Malformed or missing parameters.
401 Unauthorized	Missing, invalid, or expired bearer token. Client should refresh or re-login.
403 Forbidden	Authenticated but not permitted. CSRF failure on portal.
404 Not Found	Resource does not exist or is not visible to the caller.
409 Conflict	State collision (duplicate unique value, closed thread).
429 Too Many Requests	Lockout active. Body includes retry_after_seconds.
500 Internal Server Error	Unhandled exception; check the server log.
502/503/504	Upstream/proxy issue; not emitted by Flask itself.

APPENDIX E

Glossary of Terms

Term	Meaning
BAA	Business Associate Agreement. Required under HIPAA between a covered entity and any vendor th
Break-the-glass	Time-boxed, logged override of normal access control. See EmergencyAccessGrantRec.
Covered Entity	Under HIPAA, a health plan, healthcare clearinghouse, or healthcare provider that electronically tran
DDI	Drug-Drug Interaction. Checked via MedicationInteraction pairs when a prescription is created.
DEA Schedule	US Drug Enforcement Administration classification (I–V) of a controlled substance. Stored on Medica
DPAPI	Windows Data Protection API. Used to encrypt tokens in the Windows client.
Fernet	Symmetric encryption scheme combining AES-128-CBC with HMAC-SHA256. Used for field-level er
FHIR R4	Fast Healthcare Interoperability Resources, Release 4. An HL7 standard; MedPharm exposes a sma
JWT	JSON Web Token. MedPharm implements a narrow, handwritten HMAC-SHA256 variant.
LOINC	Logical Observation Identifiers Names and Codes. Used on LabOrder and LabResult.
NDC	National Drug Code. Ten- or eleven-digit identifier for a specific drug product.
PHI	Protected Health Information. Anything that can identify a patient plus a health fact.
POCO	Plain Old C# Object. Used to describe simple data carriers in the Windows client.
PRG	POST / Redirect / GET. Web pattern that prevents form resubmission on refresh.
TOTP	Time-based One-Time Password. RFC 6238. Basis of MFA.
WAL	SQLite Write-Ahead Logging mode. Preferred over the default rollback-journal for concurrent reads.

APPENDIX F

Revision History and Support

This volume is a living document. Each substantive revision is numbered against the MedPharm release with which it shipped, followed by a letter marking revisions of the manual itself within that release.

Revision history

Revision	Date	Author	Summary
1.7.6-D	27 May 2026	R. Stillwell	Documents the medications-catalogue bulk loaders (database/loa
1.7.6-C	27 Apr 2026	R. Stillwell	Documented the source-tree auto-update mechanism (./update.sh
1.7.6-B	26 Apr 2026	R. Stillwell	Documented cross-platform builds for the iOS / macOS clients fro
1.7.6-A	25 Apr 2026	R. Stillwell	Reissued for the 1.7.6 product line. Noted the PDF rendering fixes
1.7.5-A	20 Apr 2026	R. Stillwell	Initial issue of the developer manual.

Support contacts

Channel	Address	Use for
Email (general)	Andrew.Stillwell@enlightec.com	Questions, clarifications, corrections
Email (security)	Andrew.Stillwell@enlightec.com (prefix [security])	Suspected security vulnerabilities
Email (manual updates)	Andrew.Stillwell@enlightec.com (prefix [devmanual])	Manual errata and expansion requests
GitHub Issues	https://github.com/stillwell/MedPharm/issues	Non-confidential bug reports, feature requests
Docker Hub	https://hub.docker.com/u/enlightec	Official container images
Company	https://www.enlightec.com	Enlightec Ltd. (publisher)

How to contribute to this manual

The manual is generated by `docs/generate_developer_manual.py`. To propose a change, edit that file, run it to regenerate the PDF, verify the output opens cleanly in Adobe Acrobat Reader (or any modern PDF viewer), and submit a pull request. For substantive structural changes, discuss by email (Andrew.Stillwell@enlightec.com) before investing significant time — the editorial decisions about part ordering, signal-word usage, and level of detail are coordinated across all three volumes.

Document integrity

Each authoritative revision is hashed and the hash published alongside the PDF on the MedPharm release page. Engineers downloading the manual may verify that their copy matches the authoritative version. The hash algorithm is SHA-256 over the raw PDF bytes.

End of the MedPharm ERP Developer Manual, Volume III, Revision 1.7.6-D.